SCHOOL OF ICT, MEDIA AND ENGINEERING

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

PROJECT TITLE: BLOCKCHAIN ENABLED VOTING SYSTEM

PRESENTED BY: LEVIAN AMOS

REG NO: BCS-05-0164/2022

THIS PROJECT IS SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE ZETECH UNIVERSITY AWARD OF BACHELOR OF SCIENCE IN COMPUTER SCIENCE

MARCH, 2026

DECLARATION

I, Levian Amos, declare that this project report titled “Block chain Enabled Voting System” is my original work and has not been submitted elsewhere for any academic award. All sources of information used in this report have been duly acknowledged through proper citations and references in accordance with academic integrity standards.

Signature:

_____

Name:_____

_____

ID No: _____

Date: _____

SUPERVISOR

I ……., undersigned, do hereby certify that this is a true report for the project undertaken by the above-named student under my supervision and that it has been submitted to Zetech University with my approval

Signature…….

Date…….

ACKNOWLEDGEMENTS

I extend my deepest gratitude to Almighty God for His grace, wisdom, and unwavering guidance throughout this academic journey. My sincere appreciation goes to my project supervisor,

…….., for his invaluable mentorship, technical insights, and steadfast support from the inception to the completion of this project. His constructive feedback and encouragement were instrumental in shaping the quality and direction of this work.

I am profoundly grateful to my lecturers whose collaboration, brainstorming sessions, and moral support made this endeavor not only successful but also enjoyable. Their technical contributions and shared commitment to excellence were a constant source of motivation.

ABSTRACT

Electoral integrity is a fundamental requirement of democratic governance, yet existing voting systems at both government and institutional levels in Kenya continue to suffer from transparency deficits, susceptibility to manipulation, and lack of independent verifiability. This research presents BVote, a blockchain-based decentralized voting system designed to address these challenges through the application of Ethereum-compatible smart contract technology deployed on the Polygon blockchain network. The research adopts a phased approach in which the system is first designed, implemented, and evaluated within the institutional election context of Zetech University as a proof of concept for eventual government-scale deployment.

The research employed the Agile software development methodology, with development organized across five iterative sprints covering smart contract development, backend API implementation, frontend application development, system integration, and comprehensive testing. The BVote system comprises three architectural layers: a Solidity smart contract deployed at address 0xd6041d8eF3F653caf1cF4Ee047D6cF73bc10AcCd on the Polygon Amoy Testnet, a Node.js Express backend application programming interface hosted on the Railway cloud platform, and a vanilla JavaScript single-page application hosted on the Netlify content delivery network and publicly accessible at https://bvote-voting.netlify.app.

The system implements role-based access control across five user roles including SuperAdmin, Admin, Voter, Auditor, and Guest, with all access control enforced at the smart contract level to prevent application-layer bypass. Voter identity is protected through SHA-256 cryptographic hashing of national identification numbers, ensuring that raw identity data is never stored on the blockchain. The system mathematically prevents double voting through an on-chain mapping that records each voter's participation per election, enforced without the possibility of administrative override.

System evaluation comprised twenty-five functional test cases, eight security test cases, seven performance measurements, and user acceptance testing with ten participants from Zetech University. All functional and security test cases produced pass results. The system achieved a System Usability Scale score of 78.5, classified as Good. An economic analysis demonstrated that a production deployment serving ten thousand voters on the Polygon

mainnet would cost approximately twenty-two United States dollars, representing a transformative cost reduction compared to conventional electoral systems.

The research concludes that blockchain technology provides a technically sound, economically viable, and practically deployable foundation for credible institutional elections, with a clear pathway to government-scale deployment subject to mobile accessibility enhancements, regulatory engagement with the Independent Electoral and Boundaries Commission of Kenya, and a binding institutional pilot election. Key recommendations include the development of a mobile-optimized interface, a guided MetaMask onboarding wizard, bulk voter registration functionality, and investigation of zero-knowledge proof integration for enhanced ballot privacy.

Keywords: *Blockchain, Smart Contract, Electronic Voting, Polygon Network, Solidity, Decentralized Application, Electoral Integrity, Kenya*

Table of Contents

CHAPTER ONE 15

INTRODUCTION 15

1.1 Background 15

1.2 Introduction 16

1.3 Statement of the Problem 17

1.4 Proposed Solution 18

1.5 Objectives 19

1.5.1 General Objective 19

1.5.2 Specific Objectives 19

1.6 Research Questions 20

1.7 Justification 21

1.8 Scope 22

CHAPTER TWO 23

LITERATURE REVIEW 23

2.1 Introduction 23

2.2 Theoretical Review and Conceptual Framework 23

2.2.1 Evolution and Limitations of Conventional Voting Systems 23

2.2.2 Blockchain Technology: Theoretical Foundations 24

2.2.3 Smart Contracts and Electoral Applications 25

2.2.4 Cryptographic Techniques for Voter Privacy 26

2.2.5 Existing Blockchain Voting Implementations 27

2.2.6 Kenyan Electoral Context and Digital Governance 28

2.3 Critique of Existing Literature 29

2.4 Summary 30

2.5 Research Gaps 31

CHAPTER THREE 32

SYSTEM METHODOLOGIES.....32
3.1 Introduction.....32
3.2 Overview of System Development Methodology.....32
3.3 Justification of Chosen Methodology.....33
3.4 Development Phases and Lifecycle.....34
3.4.1 Phase One: Requirements Elicitation and System Planning.....35
3.4.2 Phase Two: Smart Contract Development and Deployment.....35
3.4.3 Phase Three: Backend Application Programming Interface Development.....35
3.4.4 Phase Four: Frontend Application Development.....36
3.4.5 Phase Five: Integration, Testing, and Evaluation.....36
3.5 Development Tools and Technologies.....37
3.6 Summary.....38
CHAPTER FOUR.....39
SYSTEM ANALYSIS AND DESIGN.....39
4.1 Introduction.....39
4.2 Systems Development Methodology.....39
4.3 Feasibility Study.....40
4.3.1 Technical Feasibility.....40
4.3.2 Economic Feasibility.....40
4.3.3 Operational Feasibility.....41
4.3.4 Legal Feasibility.....42
4.4 Requirements Elicitation.....42
4.4.1 Data Collection Methods.....42
4.4.2 Functional Requirements.....43
4.4.3 Non-Functional Requirements.....44
4.5 Data and System Analysis.....45
4.5.1 System Users and Roles.....45

4.5.2 Context Diagram 47
4.5.3 Data Flow Diagram - Level 0 49
4.5.4 Data Flow Diagram - Level 1: Voter Registration Flow 49
4.5.5 Data Flow Diagram - Level 1: Vote Casting Flow 50
4.6 System Specification 51
4.6.1 Hardware Requirements 51
4.6.2 Software Requirements 52
4.7 System Architecture 54
4.8 Logical and Physical Design 55
4.8.1 Entity Relationship Diagram 55
4.8.2 Use Case Diagram 55
4.8.3 Sequence Diagram: Vote Casting 56
4.9 Interface Design 58
4.9.1 Homepage Interface 58
4.9.2 Voter Registration Interface 58
4.9.3 Voting Interface 58
4.9.4 Admin Panel Interface 58
4.9.5 Results Interface 59
4.10 Summary 59
CHAPTER FIVE 60
SYSTEM CODE GENERATION, TESTING, ... 60
CONCLUSIONS AND RECOMMENDATIONS 60
5.1 Introduction 60
5.2 System Code Generation 60
5.2.1 Smart Contract Layer - ChainVote. Sol 60
5.2.2 Backend Layer - Node.js API 63
5.2.3 Frontend Layer - JavaScript Application 65

5.2.4 Deployment Configuration 66
5.3 Testing 67
5.3.1 Functional Testing 67
5.3.2 Security Testing 71
5.3.3 Performance Testing 73
5.3.4 User Acceptance Testing 74
5.4 Conclusions 75
5.5 Limitations 77
5.6 Recommendations 78
5.7 Overview of the Chapter 79
REFERENCES 80
APPENDICES 83

LIST OF TABLES

Table 1: Comparison of System Development Methodologies 34
Table 2: Development Tools and Technologies 38
Table 3: Estimated Production Deployment Cost on Polygon Mainnet 41
Table 4: Functional Requirements by User Role 44
Table 5: Non-Functional Requirements 45
Table 6: Minimum Hardware Requirements 52
Table 7: Software Requirements 53
Table 8: Functional Test Cases and Results 71
Table 9: Security Test Results 73
Table 10: System Performance Test Results 73
Table 11: User Acceptance Testing Results 75
Table 12: Task completion record 86
Table 13: System usability scale 87
Table 14: Project Budget 88
Table 15: Project Work Plan 89

LIST OF FIGURES

Figure 1: BVote System Role Hierarchy 46
Figure 2: BVote System Context Diagram 48
Figure 3: BVote Level 0 Data Flow Diagram 49
Figure 4: Voter Registration Data Flow Diagram (Level 1) 50
Figure 5: Vote Casting Data Flow Diagram (Level 1) 51
Figure 6: BVote Three-Tier System Architecture 54
Figure 7: BVote Entity Relationship Diagram 55
Figure 8:BVote Use Case Diagram 56
Figure 9: Sequence Diagram for Vote Casting Process 57
Figure 10: ChainVote Smart Contract Core Data Structures 61
Figure 11: ChainVote Smart Contract Core Data Structures 62
Figure 12: ChainVote castVote Function with Integrity Checks 63
Figure 13: Blockchain Service Module Initialization 64
Figure 14: Authentication Route with Signature Verification 65
Figure 15: Frontend Wallet Connection and Authentication Function 66

ACRONYMS AND ABBREVIATIONS

ABI	Application Binary Interface
Agile	Agile Software Development Methodology
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CDN	Content Delivery Network
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DApp	Decentralized Application
ERD	Entity Relationship Diagram
EVM	Ethereum Virtual Machine
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IEBC	Independent Electoral and Boundaries Commission
JSON	JavaScript Object Notation
JWT	JSON Web Token
KUCCPS	Kenya Universities and Colleges Central Placement Service
KNHR	Kenya National Commission on Human Rights
KIEMS	Kenya Integrated Electoral Management System
NFR	Non-Functional Requirement
NIST	National Institute of Standards and Technology
POL	Polygon Native Token
REST	Representational State Transfer
RPC	Remote Procedure Call

SHA	Secure Hash Algorithm
SPA	Single-Page Application
SQL	Structured Query Language
SUS	System Usability Scale
UML	Unified Modelling Language
URL	Uniform Resource Locator
UAT	User Acceptance Testing
zk- SNARK	Zero-Knowledge Succinct Non-Interactive Argument of Knowledge

DEFINITION OF TERMS

Blockchain

A distributed, append-only digital ledger in which records are grouped into cryptographically linked blocks replicated across a network of independent nodes, making historical records practically immutable.

Smart Contract

A self-executing program stored on a blockchain that automatically enforces the terms of an agreement when predefined conditions are met, without the possibility of modification or override after deployment.

Solidity

A statically typed, contract-oriented programming language designed for developing smart contracts on the Ethereum Virtual Machine and compatible networks including Polygon.

Polygon Network

An Ethereum-compatible Layer 2 blockchain scaling solution that provides high transaction throughput and low transaction fees while anchoring its security to the Ethereum mainnet.

MetaMask

A browser extension and mobile application that functions as a cryptocurrency wallet, enabling users to manage blockchain accounts and sign transactions for decentralized applications.

SHA-256

A cryptographic hash function that produces a fixed 256-bit output from an input of any length; it is computationally infeasible to reverse the function or find two inputs that produce the same output.

JSON Web Token

A compact, digitally signed token used to securely transmit user identity and role information between a client and a server, enabling stateless authentication in web applications.

Wallet Address

A unique identifier derived from a cryptographic public key that represents a user's account on a blockchain network, analogous to a bank account number.

Gas Fee

A small amount of cryptocurrency paid to compensate the network validators who process and record transactions on a blockchain network.

Testnet

A blockchain network that replicates the functionality of a main network but uses tokens with no real monetary value, used by developers for testing applications without financial risk.

Decentralized Application

A software application that runs on a distributed blockchain network rather than centralized servers, with its business logic encoded in smart contracts that cannot be altered after deployment.

Role-Based Access Control

A security mechanism in which access to system functions is granted based on the role assigned to a user, rather than on individual user permissions.

CHAPTER ONE

INTRODUCTION

1.1 Background

Democratic governance fundamentally depends on the integrity of electoral processes. Elections serve as the primary mechanism through which citizens exercise their sovereign right to determine their leadership, and any compromise to the integrity of this process directly undermines democratic governance. Globally, governments and electoral bodies have long grappled with the challenges of conducting elections that are simultaneously secure, transparent, accessible, and cost-effective. The credibility of election results depends not only on the actual conduct of the election but equally on the ability of all stakeholders to independently verify that the process was conducted fairly.

Across the world, traditional paper-based voting systems have been the dominant mode of conducting elections for centuries. However, these systems are increasingly recognized as inadequate in the modern era due to their susceptibility to various forms of manipulation including ballot stuffing, result falsification during tallying, voter impersonation, and destruction of ballot materials. The 2017 Kenyan general election, for example, was nullified by the Supreme Court of Kenya partly on grounds of irregularities in the electronic transmission of results, highlighting the vulnerability of centralized electoral systems even when electronic components are incorporated. Such incidents erode public confidence in democratic institutions and raise fundamental questions about the reliability of existing voting infrastructure.

In response to these challenges, governments and researchers worldwide have explored electronic voting as an alternative to paper-based systems. Electronic voting systems have been deployed in countries including Estonia, Brazil, India, and the United States. Estonia stands out as a global leader, having successfully conducted binding internet-based national elections since 2005. However, even these advanced electronic systems have faced criticism for relying on centralized servers that constitute single points of failure and are potentially vulnerable to cyber-attacks, insider manipulation, and lack of independent verifiability by ordinary citizens.

The emergence of blockchain technology, first introduced through the Bitcoin protocol by Satoshi Nakamoto in 2008, has opened a new frontier for the design of voting systems. A block chain is a distributed, append-only ledger in which transactions are grouped into cryptographically linked blocks and replicated across a network of nodes. The decentralized architecture of block chain eliminates reliance on any single trusted authority, while its immutability ensures that recorded data cannot be retroactively altered. These properties make block chain technology a natural candidate for application in electoral systems, where the elimination of centralized control and the guarantee of data integrity are paramount requirements.

In Kenya, the Independent Electoral and Boundaries Commission has made successive efforts to digitize the electoral process, including the introduction of the Kenya Integrated Electoral Management System and biometric voter registration. Despite these efforts, Kenya's electoral process continues to face credibility challenges, with post-election disputes becoming a recurring feature of the democratic cycle. This research proposes that a block chain-based voting system, piloted within institutional settings such as universities, can provide the empirical foundation for a government-scale decentralized electoral solution.

1.2 Introduction

This chapter introduces the research project BVote, a blockchain-based decentralized voting system designed with the ultimate objective of providing a credible, transparent, and tamper-proof electoral platform suitable for government-scale elections. Recognizing the complexity and risk associated with direct deployment in national elections, this research adopts a phased approach in which the system is first designed, implemented, and evaluated within the controlled environment of university elections, specifically targeting institutions such as Zetech University. The lessons learned and evidence gathered from institutional deployment form the empirical basis for recommendations regarding broader government adoption.

The research area of blockchain-based e-voting sits at the intersection of distributed systems, cryptography, software engineering, and political science. It addresses a globally recognized problem: how to conduct elections that are simultaneously verifiable by the

public, resistant to manipulation by insiders, protective of individual voter privacy, and accessible to ordinary citizens without requiring specialized technical knowledge. These requirements are in tension with one another, and finding a balanced technical solution represents a significant research and engineering challenge.

BVote is a full-stack decentralized application built on three architectural layers. The foundation is a Solidity smart contract deployed on the Polygon blockchain network, which encodes and enforces all election rules programmatically without the possibility of human override after deployment. The second layer is a Node.js backend application programming interface that serves as a bridge between the user-facing application and the blockchain, handling authentication, data formatting, and transaction management. The third layer is a web-based frontend application through which voters, administrators, and auditors interact with the system. Together, these three layers constitute a complete, deployable voting system that has been tested in a live blockchain environment.

The choice of university elections as the initial deployment context is deliberate and strategic. University elections share many structural characteristics with government elections, including the need for voter registration and verification, candidate nomination, ballot casting, result tallying, and public announcement of results. At the same time, the consequences of errors or failures in a university election, while significant, are less severe than in a national election, making the university context an appropriate environment for pilot testing. A successful deployment at Zetech University and similar institutions will generate the empirical evidence needed to support the case for government adoption.

1.3 Statement of the Problem

Kenya and many other developing nations face persistent and recurring challenges in the conduct of credible elections at all levels of government. The centralized architecture of existing electoral management systems creates multiple points of vulnerability that have been exploited to compromise election integrity. According to the African Union Election Observation Mission report on the 2022 Kenyan General Election, concerns regarding the transparency of result transmission and the integrity of the electoral management system were among the primary factors that undermined public confidence in the electoral process. Similar concerns have been documented in elections across Sub-Saharan Africa, where

allegations of result manipulation at tallying centers remain a common source of post-election disputes.

Existing electronic voting solutions, while representing an improvement over purely manual systems, fundamentally replicate the centralization problem in digital form. These systems store election data in centralized databases controlled by electoral authorities or technology vendors, neither of whom can be independently audited by ordinary citizens or opposition parties. The opacity of these systems means that voters have no means of independently verifying that their votes were correctly recorded, that the tally is accurate, or that the declared results reflect the actual votes cast. This absence of independent verifiability is a fundamental democratic deficit that cannot be resolved through procedural safeguards alone.

At the institutional level, university and college elections in Kenya are similarly plagued by integrity challenges. A survey conducted by the Kenya Universities and Colleges Central Placement Service (2023) found that over 70% of student union elections in Kenyan public universities in the preceding five years had been accompanied by disputes, with the most common complaints being allegations of ballot stuffing, biased counting, and results that did not reflect the actual votes cast. These disputes have on multiple occasions resulted in student unrest that disrupted academic activities, caused property damage, and in some cases resulted in injuries.

The fundamental problem that this research addresses is therefore the absence of a voting system architecture that is simultaneously decentralized, transparent, verifiable, cost-effective, and practically deployable in the Kenyan context. Existing blockchain voting research has largely remained theoretical or has been implemented in the context of developed country electoral systems with different technological and resource constraints. This research addresses the gap by developing, deploying, and evaluating a blockchain voting system specifically designed for the Kenyan institutional context, with a clear pathway to government-scale deployment.

1.4 Proposed Solution

This research proposes the investigation and development of BVote, a decentralized blockchain-based voting system designed to address the transparency, integrity, and

verifiability deficits of existing electoral systems. The research seeks to determine whether blockchain technology, specifically Ethereum-compatible smart contracts deployed on the Polygon network, can provide a technically sound and practically deployable foundation for credible elections in the Kenyan context, beginning with institutional elections at the university level as a proof of concept for eventual government-scale application.

The proposed solution is grounded in the principle that election integrity should be enforced by mathematics and cryptography rather than by trust in human administrators. In BVote, all election rules including voter eligibility verification, the prevention of double voting, vote recording, and result tallying are encoded in an immutable smart contract on the Polygon blockchain. Once deployed, these rules cannot be altered by any party, including the system developer, ensuring that the election proceeds exactly as programmed regardless of any attempt at interference. Every action taken within the system, from voter registration to vote casting, generates an immutable on-chain record that any party can independently verify.

The solution is designed with a clear scalability pathway. The pilot deployment at Zetech University targets a voter population of up to ten thousand registered students, a scale sufficient to test the system's performance, usability, and security under realistic conditions. The architectural choices made in the design of BVote, particularly the use of the Polygon network with its high transaction throughput and low fees, ensure that the system can scale to accommodate millions of voters at a cost that is feasible for government deployment. The research documents these scalability considerations explicitly to support the transition from institutional pilot to government application.

1.5 Objectives

1.5.1 General Objective

The general objective of this research is to design, develop, and evaluate a blockchain-based decentralized voting system that addresses the transparency and integrity challenges of existing electoral systems, piloted within university institutional elections as a foundation for government-scale electoral application.

1.5.2 Specific Objectives

1. To investigate the transparency, integrity, and verifiability challenges of existing government and institutional voting systems in Kenya and identify the technical requirements for a blockchain-based electoral solution.
2. To design and implement a decentralized voting system using Solidity smart contracts on the Polygon blockchain network incorporating role-based access control, cryptographic voter identity protection, and immutable vote recording.
3. To develop and deploy a full-stack application comprising a Node.js backend application programming interface and a web-based frontend interface that enables voters, administrators, and auditors to interact with the blockchain voting system.
4. To pilot the BVote system within an institutional election context at Zetech University and evaluate its performance, security, usability, and scalability as a proof of concept for government-scale electoral deployment.

1.6 Research Questions

1. What are the key transparency, integrity, and verifiability challenges of existing government and institutional voting systems in Kenya, and how can blockchain technology technically address these challenges?
2. How can Solidity smart contracts be designed and deployed on the Polygon blockchain network to enforce election integrity, prevent double voting, protect voter identity, and provide publicly verifiable election results?
3. What system architecture and implementation approach is most effective for developing a full-stack blockchain voting application that is secure, usable, and accessible to institutional voters and administrators without requiring blockchain technical knowledge?
4. To what extent does the BVote system demonstrate the technical feasibility and practical readiness of blockchain-based voting for institutional deployment, and what conditions must be met for scaling to government-level elections?

1.7 Justification

The justification for this research is rooted in the critical importance of election integrity to democratic governance and the documented inadequacy of existing electoral systems in Kenya and comparable developing nations. The recurring pattern of electoral disputes, result manipulation allegations, and post-election unrest that characterizes the Kenyan electoral landscape represents a significant threat to democratic stability that existing technical solutions have failed to adequately address. This research directly responds to this challenge by investigating and demonstrating a technically superior alternative.

From a governance perspective, the successful development and piloting of BVote contributes to the national discourse on electoral reform by providing concrete technical evidence of the feasibility of decentralized, blockchain-based electoral infrastructure. The Independent Electoral and Boundaries Commission of Kenya, along with electoral bodies in comparable nations, requires empirically grounded evidence of the reliability, security, and scalability of blockchain voting systems before considering adoption. This research generates that evidence through the pilot deployment at Zetech University.

The research benefits a broad range of stakeholders. Students and staff at Zetech University benefit directly from participation in a more transparent and credible election process. The broader Kenyan student population benefits from the documentation of a replicable institutional voting model that other universities can adopt. Government stakeholders and electoral reform advocates benefit from the research findings regarding the scalability and cost-effectiveness of blockchain voting at the national level. The academic community benefits from a rigorous evaluation of blockchain voting in the East African context, a geography that is underrepresented in the existing literature on e-voting systems.

The choice of the Polygon blockchain network is specifically justified by the economic realities of government deployment in developing countries. Unlike the Ethereum mainnet where a single transaction can cost several dollars, Polygon transaction fees are measured in fractions of a cent, making a national election involving millions of voters economically feasible on a government budget. This research demonstrates and documents this cost advantage with empirical data from the institutional pilot deployment, providing a financial case for government adoption alongside the technical case.

1.8 Scope

This research focuses on the design, development, deployment, and evaluation of a blockchain-based voting system with the ultimate research goal of informing the development of a government-scale electoral solution. The immediate scope of the research is confined to the institutional election context, specifically the student leadership elections at Zetech University, which serves as the primary pilot site for system testing and evaluation. The findings from this pilot are used to draw conclusions about the broader applicability of the system to government elections.

The geographical scope of the research is Kenya, with specific attention to the regulatory, infrastructural, and social context of Kenyan electoral processes. The system is designed with awareness of the Kenya Elections Act and the technical standards prescribed by the Independent Electoral and Boundaries Commission, ensuring that the research findings are relevant to the Kenyan legal and regulatory framework. The research does not, however, claim legal compliance with electoral regulations, as formal compliance assessment is beyond the scope of an academic research project and would require engagement with regulatory authorities.

The technical scope of the research encompasses the complete development of a three-layer system architecture comprising a Solidity smart contract on the Polygon Amoy Testnet, a Node.js Express backend application programming interface, and a web-based vanilla JavaScript frontend application. The voter population for the pilot deployment is limited to a maximum of ten thousand registered voters, consistent with the scale of Zetech University student elections. The research does not extend to biometric voter identification, integration with the national integrated identity management system, or deployment on the Polygon mainnet, all of which are identified as essential components of a full government deployment and are recommended as priorities for future research.

The temporal scope of the research is confined to the duration of the academic project period. The system evaluation is based on testing conducted within this period, including functional testing, security testing, and user acceptance testing. Long-term reliability assessment, electoral cycle testing across multiple elections, and longitudinal user studies are beyond the scope of this research but are identified as important areas for future investigation to support the transition from institutional pilot to government

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter presents a review of existing literature relevant to the research problem of developing a blockchain-based voting system for institutional and government elections. The review is organized thematically around the key variables and concepts that underpin the research: the evolution and limitations of conventional voting systems, the theoretical foundations of blockchain technology, and the application of smart contracts in electoral systems, cryptographic techniques for voter privacy, existing blockchain voting implementations and their limitations, and the specific electoral context of Kenya. The chapter concludes with a critique of the existing literature, a summary of key findings, and an identification of the research gaps that this study seeks to address. All sources cited in this chapter are drawn from peer-reviewed journals, conference proceedings, and authoritative institutional reports published within the last five years, except where older foundational works are referenced.

2.2 Theoretical Review and Conceptual Framework

2.2.1 Evolution and Limitations of Conventional Voting Systems

The history of voting systems reflects a continuous effort to balance the competing requirements of accessibility, security, and verifiability. Paper-based voting systems, which remain dominant in most of the world including Kenya, offer the advantage of simplicity and a physical audit trail. However, they are vulnerable to a range of integrity threats including ballot box stuffing, miscounting, fraudulent result alteration during transmission, and voter impersonation. Norris (2019) documents that globally, over 40% of elections held between 2012 and 2018 were accompanied by allegations of fraud, with result manipulation during tallying being the most frequently cited form of irregularity. These figures underscore the systemic nature of the problem and the inadequacy of procedural safeguards alone in guaranteeing electoral integrity. Electronic voting systems, introduced as a

technological solution to paper-based vulnerabilities, have demonstrated mixed results. Direct-recording electronic voting machines, as used in parts of the United States and India, eliminate the risk of physical ballot tampering but introduce new concerns about software integrity and the absence of a verifiable paper trail. Springall et al. (2014) demonstrated critical security vulnerabilities in Estonia's internet voting system that could have allowed a sophisticated attacker to alter votes undetected, highlighting the risks associated with centralized electronic systems even in technologically advanced implementations. The fundamental limitation of all centralized electronic voting systems is that their integrity depends on trust in the system operators, a requirement that is incompatible with the adversarial environment of contested elections.

In Kenya specifically, the trajectory of electronic voting adoption has been marked by controversy. The introduction of the Kenya Integrated Electoral Management System, which was intended to provide real-time result transmission from polling stations, became the focal point of the 2017 election dispute that led to the first successful nullification of a presidential election in African history. The Supreme Court of Kenya found that the system had been compromised and that results transmitted through it could not be relied upon as accurate. Orago (2018) argues that the fundamental design flaw of the Kenyan electronic voting system was its centralization, which created a single point of vulnerability that was exploited to cast doubt on the entire electoral process. This finding directly motivates the decentralized architecture proposed in BVote.

2.2.2 Blockchain Technology: Theoretical Foundations

Blockchain technology was introduced by Nakamoto (2008) as the foundational infrastructure for the Bitcoin cryptocurrency. At its core, a blockchain is a distributed ledger in which records, called transactions, are grouped into blocks that are cryptographically linked in a sequential chain. Each block contains a cryptographic hash of the preceding block, a timestamp, and a set of transactions. The distributed nature of the blockchain means that identical copies of the ledger are maintained by multiple independent nodes, and any attempt to alter a historical record would require the simultaneous alteration of the majority of all copies, a computational feat that is practically infeasible in a sufficiently decentralized network.

The consensus mechanisms that underpin blockchain networks are central to their security properties. Proof of Work, used in Bitcoin, requires nodes to expend computational

resources to validate new blocks, making the cost of attacking the network prohibitively high. Proof of Stake, used in Ethereum and the Polygon network that underpins BVote, requires validators to stake cryptocurrency as collateral, which is forfeited if they attempt to validate fraudulent transactions. Saleh (2021) provides a formal economic analysis demonstrating that Proof of Stake mechanisms achieve equivalent security guarantees to Proof of Work at a fraction of the energy cost, making them more suitable for applications requiring frequent transactions at low cost, such as a national voting system.

The Polygon network, chosen as the deployment platform for BVote, is an Ethereum-compatible Layer 2 scaling solution that achieves transaction throughput of up to 65,000 transactions per second and average transaction fees of less than 0.01 United States dollars. Matic Network (2021) describes the Polygon architecture as a commit chain that periodically anchors its state to the Ethereum mainnet, inheriting Ethereum's security guarantees while achieving significantly higher performance and lower cost. For a voting application serving millions of voters, these performance characteristics are essential. A national election in Kenya involving approximately twenty-two million registered voters would require the processing of twenty-two million vote transactions, a workload that the Polygon network can accommodate within a matter of hours at a total transaction cost estimated at less than two thousand United States dollars.

2.2.3 Smart Contracts and Electoral Applications

Smart contracts, introduced conceptually by Szabo (1994) and implemented practically on the Ethereum platform, are self-executing programs stored on the blockchain that automatically enforce the terms of an agreement when predefined conditions are met. In the context of voting systems, smart contracts can encode the complete set of electoral rules, including voter eligibility verification, the prevention of double voting, vote recording, result tallying, and access control, in a form that is automatically enforced without the possibility of human override. Once deployed, a smart contract cannot be modified, ensuring that the rules governing an election cannot be changed after the election has begun.

Wang et al. (2022) conducted a systematic review of blockchain-based voting systems and identified smart contract-based access control as the most effective mechanism for preventing unauthorized access to voting functions. Their analysis of twenty-three blockchain voting implementations found that systems using role-based access control

enforced by smart contracts had zero reported instances of unauthorized vote manipulation, compared to a 23% incidence of access control violations in systems relying solely on application-level controls. This finding directly informs the design of BVote, which implements role-based access control at the smart contract level, ensuring that voting privileges cannot be circumvented through manipulation of the application layer.

The Solidity programming language, used to develop the BVote smart contract, is the dominant language for Ethereum-compatible smart contract development. Antonopoulos and Wood (2018) describe Solidity as a statically typed, contract-oriented language specifically designed for the Ethereum Virtual Machine, with native support for the data structures and patterns required in financial and governance applications. The BVote contract implements five core data structures: a Voter struct mapping wallet addresses to voter credentials and status, a Candidate struct storing candidate information, an Election struct encoding election parameters and state, a vote recording mapping, and a role management mapping for access control. These structures collectively encode the complete state of the electoral system on the blockchain.

2.2.4 Cryptographic Techniques for Voter Privacy

A fundamental tension in the design of transparent voting systems is the requirement to simultaneously provide public verifiability of the aggregate result and individual privacy of each voter's specific ballot choice. In a fully transparent blockchain system where all transactions are publicly visible, naive implementation would allow any observer to determine how each voter voted by associating their wallet address with their vote transaction. Protecting voter privacy while maintaining result transparency requires the application of cryptographic techniques.

Hashing functions play a central role in voter identity protection in BVote. The system applies the SHA-256 cryptographic hash function to each voter's national identification number before recording it on the blockchain. SHA-256 is a one-way function, meaning that while it is computationally trivial to compute the hash of a given input, it is computationally infeasible to recover the original input from its hash. Dang (2015), in the National Institute of Standards and Technology documentation of SHA-256, characterizes the function as collision-resistant, meaning that it is computationally infeasible to find two different inputs that produce the same hash output. This property ensures that the national identification numbers of voters cannot be recovered from the blockchain records,

protecting their identity while still enabling verification that no individual has registered twice.

Zero-knowledge proofs represent a more advanced cryptographic technique for voter privacy that has been explored in academic blockchain voting research. A zero-knowledge proof allows a prover to demonstrate to a verifier that a statement is true without revealing any information beyond the truth of the statement itself. Ben-Sasson et al. (2018) demonstrate the application of zk-SNARKs, a class of zero-knowledge proofs, to blockchain voting systems to enable complete anonymization of vote-to-voter associations. While BVote does not implement zero-knowledge proofs in its current form due to the added implementation complexity, the system architecture is designed to be compatible with future integration of such techniques as part of the pathway to government-scale deployment.

2.2.5 Existing Blockchain Voting Implementations

Several blockchain-based voting systems have been proposed and implemented in academic and commercial contexts, each offering insights relevant to the design of BVote. Hjálmarsson et al. (2018) proposed a blockchain-based e-voting system using Ethereum smart contracts and demonstrated its technical feasibility through a proof-of-concept implementation. Their system achieved the core properties of immutability, transparency, and decentralization but was found to have scalability limitations due to the high transaction costs on the Ethereum mainnet, a limitation that BVote addresses through the use of the Polygon network.

Ayed (2017) proposed a conceptual blockchain voting model that incorporated end-to-end verifiability, allowing voters to verify that their individual votes were correctly included in the final tally. While Ayed's model remained theoretical, it identified the key requirements for a practical blockchain voting system that informed subsequent implementations including BVote. Particularly relevant to this research is Ayed's finding that the decentralized nature of blockchain fundamentally resolves the trusted authority problem that undermines conventional electronic voting systems, as the correctness of the tally is guaranteed by the mathematical properties of the blockchain rather than by trust in any human operator.

The Follow My Vote platform, a commercial blockchain voting application, demonstrated the practical feasibility of blockchain voting in small-scale elections but faced criticism for

its reliance on a proprietary blockchain rather than a public, permissionless network. McCorry et al. (2017) argue that voting systems built on private or consortium blockchains fail to achieve the decentralization properties that are the primary security advantage of blockchain technology, as the operators of the private blockchain remain a trusted authority that could potentially manipulate the results. BVote addresses this concern through its deployment on the public Polygon network, where no single entity controls the blockchain infrastructure.

Tarasov and Tewari (2017) evaluated the performance of blockchain voting systems under realistic election conditions and found that the primary bottleneck was transaction throughput during peak voting periods, when large numbers of voters attempt to cast ballots simultaneously. Their simulation of a national election scenario found that the Ethereum mainnet, with a throughput of approximately 15 transactions per second, would require over 17 days to process the votes of a national electorate of twenty million voters. This finding reinforces the importance of the Polygon network choice in BVote, where throughput of up to 65,000 transactions per second reduces the processing time for an equivalent electorate to approximately five minutes.

2.2.6 Kenyan Electoral Context and Digital Governance

The Kenyan electoral context presents specific challenges and opportunities for the adoption of blockchain-based voting. Kenya has a relatively high mobile phone penetration rate of approximately 115% and an internet penetration rate of approximately 43% as of 2023, according to the Communications Authority of Kenya. While these figures suggest a foundation for digital service adoption, the significant proportion of the population without reliable internet access presents a challenge for any internet-based voting system. This research acknowledges this limitation and positions BVote as a medium-term solution appropriate for urban and peri-urban populations with reliable connectivity, with recommendations for offline-capable extensions as part of the pathway to universal government deployment.

The legal framework for electronic voting in Kenya is established primarily by the Elections Act (2011) and the Elections (General) Regulations (2012), which provide for the use of electronic devices in voter registration, identification, and result transmission but do not explicitly address blockchain-based voting. Odote and Musumba (2020) analyze the Kenyan legal framework for e-voting and conclude that while the current legislation does

not prohibit blockchain voting, regulatory clarification and potential legislative amendment would be required before a blockchain voting system could be used in a binding national election. This research acknowledges this regulatory gap and positions BVote as a technical proof of concept that demonstrates feasibility, with the expectation that regulatory adaptation would precede formal government adoption.

2.3 Critique of Existing Literature

The existing literature on blockchain-based voting systems demonstrates clear technical feasibility but reveals several significant gaps that limit its applicability to the Kenyan context and to practical government deployment. The most pervasive limitation is the dominance of theoretical or simulation-based research that does not involve actual deployment and testing on a live public blockchain. Many of the systems reviewed by Wang et al. (2022) were evaluated through simulations or private testnet deployments that do not capture the full complexity of real-world conditions including network congestion, variable transaction fees, and user behaviour under authentic election conditions. BVote addresses this gap through its deployment on the Polygon Amoy public testnet and its evaluation through actual transaction processing.

A second significant limitation of existing research is the neglect of the user experience dimension of blockchain voting systems. The majority of academic implementations require users to interact directly with blockchain interfaces, wallet management systems, and cryptographic keys, a level of technical sophistication that is unrealistic for the general voting population. Khoury et al. (2021) found that in user testing of blockchain voting prototypes, over 65% of participants reported difficulty with the wallet setup and transaction signing processes. BVote addresses this limitation through a carefully designed frontend application that abstracts blockchain complexity behind a familiar web interface, requiring only the installation of the MetaMask browser extension, which is increasingly mainstream.

The existing literature also largely neglects the developing country context, with the majority of implementations and evaluations conducted in the context of European or North American electoral systems with different infrastructural, economic, and regulatory environments. The specific challenges of the Kenyan context, including variable internet

connectivity, the dominance of mobile internet access over desktop access, the legal framework of the Elections Act, and the cost sensitivity of government procurement, are absent from existing research. This research directly addresses this gap by designing BVote specifically for the Kenyan institutional context and evaluating it against the requirements of eventual Kenyan government deployment.

2.4 Summary

This chapter has reviewed the theoretical and empirical literature relevant to the development of a blockchain-based voting system for institutional and government elections. The review established that conventional voting systems, both paper-based and electronic, suffer from fundamental integrity limitations rooted in their centralized architectures and dependence on trusted human administrators. Blockchain technology, through its properties of decentralization, immutability, and cryptographic security, offers a theoretically sound foundation for addressing these limitations. Smart contracts enable the programmatic enforcement of election rules without the possibility of human override, while cryptographic hashing techniques provide voter identity protection. Existing blockchain voting implementations have demonstrated technical feasibility but have been limited by scalability constraints on high-cost networks, insufficient attention to user experience, and a lack of empirical evaluation in real-world deployment conditions.

The Polygon network addresses the scalability and cost limitations of earlier Ethereum-based implementations, providing the transaction throughput and fee structure required for national-scale deployment. The Kenyan electoral context, characterized by persistent integrity challenges, a supportive but incomplete regulatory framework, and growing digital infrastructure, provides both the motivation and the practical environment for the BVote pilot deployment. The literature review confirms that BVote occupies a meaningful position in the research landscape by combining a public blockchain deployment, a user-accessible interface design, and a developing-country institutional pilot that collectively address the most significant gaps in existing research.

2.5 Research Gaps

The review of existing literature identifies the following specific research gaps that this study addresses. First, there is a lack of empirical research on blockchain voting systems deployed and tested on public blockchain networks under realistic election conditions, as opposed to theoretical models or private testnet simulations. This research addresses this gap through the live deployment and testing of BVote on the Polygon Amoy public testnet. Second, existing research insufficiently addresses the user experience requirements of non-technical voters, focusing predominantly on cryptographic and architectural properties while neglecting usability. This research addresses this gap through the development of a user-accessible web interface evaluated through user acceptance testing. Third, there is a near-complete absence of blockchain voting research conducted in the Sub-Saharan African context, particularly in Kenya, where unique electoral integrity challenges and regulatory conditions apply. This research addresses this gap through the Zetech University pilot deployment and the explicit framing of BVote within the Kenyan regulatory and infrastructural context. Fourth, existing research does not provide empirically grounded cost analyses of blockchain voting at national scale, limiting the ability of governments to assess the economic feasibility of adoption. This research addresses this gap through documented cost analysis of the BVote deployment extrapolated to national election scale using the Polygon network.

CHAPTER THREE

SYSTEM METHODOLOGIES

3.1 Introduction

This chapter describes the system development methodology adopted for the BVote blockchain-based voting system. It provides an outline of the selected methodology, justifies the choice relative to alternative approaches, and describes how the methodology was applied across the development lifecycle of the research project. The chapter also presents the phases of the development process and explains how each phase contributed to the realization of the system objectives stated in Chapter One. The methodology described in this chapter provides the structural framework within which the system analysis, design, implementation, and testing activities documented in subsequent chapters were conducted.

3.2 Overview of System Development Methodology

A system development methodology is a structured framework that guides the planning, design, implementation, and evaluation of a software system. The choice of methodology has significant implications for the organization of development activities, the management of requirements changes, the timing of testing activities, and the quality of the final system. Different methodologies are suited to different project contexts depending on factors such as the clarity and stability of requirements, the complexity of the system, the size of the development team, and the criticality of early delivery.

For the development of BVote, the Agile software development methodology was selected, with specific application of the Scrum framework for sprint-based iterative development. Agile methodologies are characterized by iterative and incremental development cycles, continuous stakeholder feedback, adaptive planning, and an emphasis on working software over comprehensive documentation. The Agile Manifesto, articulated by Beck et al. (2001), defines the core values of Agile development as prioritizing individuals and interactions over processes and tools, working software over comprehensive documentation, customer

collaboration over contract negotiation, and responding to change over following a fixed plan.

The development of BVote proceeded through five iterative sprints, each of approximately two weeks duration, covering the phases of requirements elicitation, smart contract development, backend development, frontend development, and integrated testing. At the conclusion of each sprint, a working increment of the system was produced and evaluated, with findings from the evaluation informing the priorities of the subsequent sprint. This iterative approach was particularly suited to the BVote project given the novelty of blockchain application development and the evolving understanding of smart contract design patterns and Polygon network characteristics that emerged during the development process.

3.3 Justification of Chosen Methodology

The selection of the agile methodology for BVote was justified by several characteristics of the project that distinguished it from contexts where traditional waterfall or spiral methodologies might be more appropriate. First, the development of a blockchain-based voting system involves significant technical uncertainty, particularly regarding smart contract design, gas optimization, and the interaction between the blockchain layer and the application layer. Agile's embrace of iterative development and continuous learning was well-suited to navigating this uncertainty, allowing the development approach to be refined as technical knowledge accumulated.

Second, the requirements of the BVote system, while broadly defined by the research objectives, required refinement through iterative prototyping and testing. The specific requirements of the voter registration workflow, the admin panel design, the role-based access control model, and the results publication mechanism were all refined through iterative development cycles informed by testing and user feedback. A waterfall methodology, which requires complete requirements specification before development begins, would have been poorly suited to this context.

Third, the parallel development of three distinct system layers, namely the smart contract, the backend application programming interface, and the frontend application, benefited from the Agile approach of delivering working increments of each layer and integrating

them progressively. This approach allowed integration issues to be identified and resolved early in the development process rather than at a final integration phase, reducing the risk of late-stage failures.

The table below presents a comparison of the Agile methodology against the Waterfall and Spiral alternatives, demonstrating the suitability of Agile for the BVote project context.

Table 3.1: Comparison of System Development Methodologies

Criterion	Waterfall	Spiral	Agile (Selected)
Requirements Flexibility	Rigid changes costly	Moderate flexibility	Highly flexible
Technical Uncertainty	Poorly suited	Moderately suited	Well suited
Delivery Approach	Single final delivery	Incremental releases	Iterative increments
Testing Integration	End of project	Per spiral cycle	Continuous
Team Size Suitability	Large teams	Medium teams	Small to medium
Documentation	Comprehensive upfront	Risk-driven	Lightweight, iterative
Suitability for BVote	Low	Moderate	High

Table 1: Comparison of System Development Methodologies

3.4 Development Phases and Lifecycle

The BVote development lifecycle was organized into five phases, each corresponding to a development sprint. The phases are described below in sequence, though in practice, some activities overlapped between phases as the iterative nature of Agile development permitted parallel work streams.

3.4.1 Phase One: Requirements Elicitation and System Planning

The first phase of the BVote development lifecycle focused on the elicitation and documentation of system requirements. This phase involved a review of existing blockchain voting literature, an analysis of the Kenyan electoral regulatory framework, interviews with student association representatives at Zetech University to understand the specific requirements of institutional elections, and a technical assessment of available blockchain platforms and development tools. The outputs of this phase included a documented set of functional and non-functional requirements, a selection of the Polygon network and Solidity smart contract language as the blockchain platform, and a high-level system architecture design.

The requirements elicitation process revealed five categories of system user, each with distinct functional requirements: the SuperAdmin responsible for overall system governance, Admins responsible for election management, Voters requiring registration and ballot casting functionality, Auditors requiring read access to election records, and Guests requiring public access to election information and results. These user categories directly informed the role-based access control model implemented in the smart contract.

3.4.2 Phase Two: Smart Contract Development and Deployment

The second phase focused on the design and implementation of the ChainVote Solidity smart contract, which forms the foundational layer of the BVote system. This phase involved the design of the contract data structures, the implementation of role-based access control functions, the development of voter registration, candidate management, election lifecycle management, and vote casting functions, and the testing of the contract using the Hardhat development framework. The contract was compiled using Solidity version 0.8.20 with optimizer settings to minimize gas consumption.

Deployment of the contract to the Polygon Amoy Testnet was accomplished using a custom deployment script developed with the ethers.js library. The deployed contract address, 0xd6041d8eF3F653caf1cF4Ee047D6cF73bc10AcCd, serves as the immutable foundation of the BVote system. All subsequent system layers interact with this contract through its application binary interface, a JSON specification of the contract's public functions and data structures generated during the compilation process.

3.4.3 Phase Three: Backend Application Programming Interface Development

The third phase involved the development of the Node.js Express backend application programming interface, which serves as the intermediary layer between the frontend application and the blockchain. This phase produced seven backend modules: the main server configuration, authentication middleware using JSON Web Tokens, and five route handlers covering public endpoints, authentication, voter functions, administrator functions, and SuperAdmin functions. The blockchain service module, which encapsulates all interactions with the deployed smart contract through the ethers.js library, was developed and tested during this phase.

The backend was designed with security as a primary consideration, incorporating the Helmet.js library for HTTP security header management, rate limiting middleware to prevent denial-of-service attacks, Cross-Origin Resource Sharing configuration to restrict API access to authorized frontend origins, and on-chain role verification for all privileged operations to prevent authorization bypass through token manipulation.

3.4.4 Phase Four: Frontend Application Development

The fourth phase involved the development of the web-based frontend application, implemented as a single-page application using vanilla HTML, CSS, and JavaScript without framework dependencies to minimize complexity and maximize compatibility. The frontend was structured as three separate files in accordance with web development best practices: an HTML structure file, a CSS stylesheet, and a JavaScript application logic file. This separation of concerns enhances maintainability and allows independent modification of presentation and behaviour.

The frontend integrates with the MetaMask browser extension for wallet management and transaction signing, abstracting the complexity of blockchain key management from the end user. The user interface provides distinct views for each user role, with navigation elements dynamically rendered based on the authenticated user's role as returned by the backend authentication endpoint. The frontend was deployed to the Netlify content delivery network for public accessibility.

3.4.5 Phase Five: Integration, Testing, and Evaluation

The fifth phase comprised the integration of the three system layers and the conduct of comprehensive system testing. Integration testing verified the correct communication between the frontend, backend, and blockchain layers across all supported user workflows. Functional testing validated each system function against its specified requirements.

Security testing assessed the system's resistance to common web application vulnerabilities and blockchain-specific attack vectors. User acceptance testing engaged representative users from the Zetech University student community to evaluate the system's usability and fitness for purpose.

The testing phase also included performance evaluation of the system under simulated load conditions to assess its suitability for deployment at the scale of a university election with up to ten thousand concurrent voters. The results of all testing activities are documented in Chapter Five of this report.

3.5 Development Tools and Technologies

The development of BVote utilized a carefully selected set of tools and technologies appropriate for each layer of the system architecture. The selection criteria emphasized open-source availability, active community support, compatibility between components, and suitability for the Polygon blockchain environment.

Table 3.2: Development Tools and Technologies

Layer	Tool / Technology	Purpose
Blockchain	Solidity 0.8.20	Smart contract programming language
Blockchain	Hardhat 3.x	Smart contract development and testing framework
Blockchain	Polygon Amoy Testnet	Public blockchain deployment environment
Blockchain	ethers.js 6.x	Blockchain interaction library
Blockchain	MetaMask	Browser wallet for transaction signing
Backend	Node.js 22.x	JavaScript runtime environment
Backend	Express.js 4.x	Web application framework
Backend	JSON Web Token	Authentication token standard

Layer	Tool / Technology	Purpose
Backend	Helmet.js	HTTP security header management
Backend	dotenv	Environment variable management
Frontend	HTML5 / CSS3	Structure and presentation
Frontend	Vanilla JavaScript	Application logic and API integration
Frontend	Google Fonts API	Typography (Playfair Display, DM Sans)
Deployment	Netlify	Frontend hosting and content delivery
Deployment	Railway	Backend hosting and process management
Version Control	Git / GitHub	Source code management and collaboration

Table 2: Development Tools and Technologies

3.6 Summary

This chapter has described the Agile system development methodology adopted for the BVote project, justified its selection over alternative methodologies, and outlined the five development phases through which the system was realized. The iterative and incremental nature of the Agile approach was demonstrated to be well suited to the technical uncertainty and evolving requirements characteristic of blockchain application development. The chapter also documented the tools and technologies employed across the three system layers. The subsequent chapter presents the detailed system analysis and design outcomes produced through the application of this methodology, including the system architecture, data flow diagrams, entity relationship models, use case specifications, and interface designs that guided the implementation of BVote.

CHAPTER FOUR

SYSTEM ANALYSIS AND DESIGN

4.1 Introduction

This chapter presents the comprehensive system analysis and design of the BVote blockchain-based voting system. The chapter covers the feasibility study conducted prior to development, the requirements elicitation process and its outcomes, the data and system analysis, the system specification, the system architecture, and the logical and physical design of the system. The design artifacts presented in this chapter, including data flow diagrams, entity relationship diagrams, use case diagrams, and interface designs, collectively constitute the blueprint from which the BVote system was implemented. The analysis and design work documented in this chapter was conducted in accordance with the Agile methodology described in Chapter Three, with design artifacts refined iteratively across the five development sprints.

4.2 Systems Development Methodology

The BVote system was developed using the Agile Scrum methodology as described in Chapter Three. In the context of system analysis and design, the Agile approach manifested as a series of iterative design cycles in which initial design artifacts were progressively refined based on implementation experience and testing feedback. The system architecture was established in the first sprint and remained stable throughout the project, while lower-level design decisions regarding specific data structures, API endpoint designs, and interface layouts were refined across subsequent sprints. This section provides a brief restatement of the methodology to contextualize the design decisions documented in subsequent sections.

The object-oriented analysis and design approach was applied throughout the BVote system design, with the smart contract, backend, and frontend layers each designed as collections of interacting objects with well-defined responsibilities and interfaces. The Unified Modelling Language version 2.x was used as the notation standard for all system

diagrams, including use case diagrams, sequence diagrams, and class diagrams, ensuring consistency and industry-standard communicability of the design documentation.

4.3 Feasibility Study

A feasibility study was conducted prior to the commencement of BVote development to assess the viability of the proposed system from technical, economic, operational, and legal perspectives. The outcomes of the feasibility study confirmed the viability of the project and informed key design decisions regarding the choice of blockchain platform, the scope of the pilot deployment, and the approach to regulatory compliance.

4.3.1 Technical Feasibility

The technical feasibility assessment evaluated whether the proposed technology stack was capable of delivering the required system functionality within the constraints of the development environment. The Polygon Amoy Testnet was assessed as technically suitable for the BVote pilot deployment, offering a fully functional Ethereum-compatible blockchain environment with free test tokens, high transaction throughput, and public block explorer support for result verification. The Solidity programming language, Hardhat development framework, Node.js runtime, and JavaScript frontend technologies were all assessed as mature, well-documented, and supported by active developer communities. The availability of the MetaMask browser extension as a mainstream wallet solution was assessed as sufficient to support the user-facing requirements of the system. The technical feasibility assessment concluded that all required capabilities were available within the selected technology stack.

4.3.2 Economic Feasibility

The economic feasibility assessment evaluated the cost of developing and deploying the BVote system relative to the benefits it delivers. The development costs were confined to developer time, as all technologies employed are open source and freely available. The deployment costs on the Polygon Amoy Testnet were zero, as the network provides free test tokens. For a production deployment on the Polygon mainnet serving ten thousand voters at Zetech University, the estimated total cost of smart contract deployment, voter registration transactions, and vote casting transactions is approximately twenty-two United States dollars, as detailed in Table 4.1 below. This cost compares extremely favourably

with the estimated cost of a conventional paper-based election at a university of ten thousand students, which typically involves printing, staffing, counting, and administrative costs in the range of fifty thousand to one hundred thousand Kenyan shillings.

Table 4.1: Estimated Production Deployment Cost on Polygon Mainnet

Transaction Type	Quantity	Estimated Cost (USD)
Smart contract deployment	1	$2.00
Voter registration (10,000 voters)	10,000	$10.00
Vote casting (10,000 votes)	10,000	$10.00
Election management transactions	~20	$0.50
TOTAL		$22.50

Table 3: Estimated Production Deployment Cost on Polygon Mainnet

4.3.3 Operational Feasibility

The operational feasibility assessment evaluated whether the BVote system would be practically usable by the intended user population of university students and administrative staff. The assessment identified that the primary operational requirement beyond standard internet access is the installation of the MetaMask browser extension, which is available free of charge for all major desktop browsers. The web-based frontend design, which requires no software installation beyond a standard browser and the MetaMask extension, was assessed as operationally feasible for the Zetech University student population, the majority of whom access internet services through personal laptops or desktop computers. The assessment identified mobile accessibility as a potential operational constraint, as the MetaMask mobile application provides a different user experience from the desktop extension, and recommended the development of a mobile-optimized interface as a priority for future development.

4.3.4 Legal Feasibility

The legal feasibility assessment evaluated the extent to which the BVote system is consistent with the Kenyan legal framework for elections. The assessment found that the Kenya Elections Act (2011) and associated regulations permit the use of electronic systems in electoral processes but do not provide specific provisions for blockchain-based voting. The assessment concluded that the BVote system, in its pilot deployment for institutional elections at Zetech University, operates within a legal context governed by the university's internal governance regulations rather than the Elections Act, and is therefore legally feasible for the pilot deployment. Formal compliance with the Elections Act would require engagement with the Independent Electoral and Boundaries Commission and potential legislative clarification, and is identified as a prerequisite for government-scale deployment.

4.4 Requirements Elicitation

4.4.1 Data Collection Methods

Requirements for the BVote system were elicited through three primary data collection methods. First, structured interviews were conducted with five members of the Zetech University Student Association executive committee to understand the current process for conducting student elections, the specific pain points and integrity concerns associated with existing methods, and the functional requirements for a digital replacement. Second, a review of the Kenya Elections Act and the Independent Electoral and Boundaries Commission technical standards was conducted to identify the regulatory requirements that any government-scale deployment would need to satisfy. Third, a comparative analysis of five existing blockchain voting research implementations was conducted to identify design patterns, common technical requirements, and pitfalls to avoid.

The interviews with student association representatives revealed that the most significant concerns regarding existing election methods were the opacity of the vote counting process, the potential for ballot papers to be added or removed after voting had closed, the difficulty of conducting elections across multiple campuses simultaneously, and the length of time required to announce results. These concerns directly informed the core functional

requirements of transparency, immutability, geographic accessibility, and real-time result availability that are addressed by the BVote design.

4.4.2 Functional Requirements

The functional requirements of the BVote system, derived from the requirements elicitation process, are organized by user role as presented in Table 4.2 below.

Table 4.2: Functional Requirements by User Role

User Role	Req. ID	Functional Requirement
SuperAdmin	FR-01	Shall be able to grant and revoke Admin role to registered wallet addresses
SuperAdmin	FR-02	Shall have all Admin privileges in addition to role management functions
Admin	FR-03	Shall be able to add candidates with name, party, and position attributes
Admin	FR-04	Shall be able to create elections with name, description, start time, end time, and candidate list
Admin	FR-05	Shall be able to open, close, and cancel elections
Admin	FR-06	Shall be able to approve or reject voter registration applications
Admin	FR-07	Shall be able to publish election results after an election is closed
Voter	FR-08	Shall be able to register using full name, national ID, and district
Voter	FR-09	Shall be able to cast exactly one vote per election after approval
Voter	FR-10	Shall be able to check their own registration status

User Role	Req. ID	Functional Requirement
Voter	FR-11	Shall receive a blockchain transaction receipt upon casting a vote
Auditor / Guest	FR-12	Shall be able to view all elections and their status without authentication
Auditor / Guest	FR-13	Shall be able to view published election results without authentication
All Users	FR-14	Shall authenticate using MetaMask wallet signature without a password
All Users	FR-15	Shall be prevented from performing actions beyond their assigned role

Table 4: Functional Requirements by User Role

4.4.3 Non-Functional Requirements

In addition to functional requirements, the following non-functional requirements were identified as essential characteristics of the BVote system.

Table 4.3: Non-Functional Requirements

Category	Req. ID	Non-Functional Requirement
Security	NFR-01	All voter national ID numbers shall be SHA-256 hashed before storage
Security	NFR-02	All API endpoints shall be protected by JSON Web Token authentication where applicable
Security	NFR-03	Role-based access control shall be enforced at both the application and smart contract levels
Integrity	NFR-04	All votes shall be recorded on the blockchain and shall be immutable once cast

Category	Req. ID	Non-Functional Requirement
Integrity	NFR-05	The system shall prevent any voter from casting more than one vote per election
Transparency	NFR-06	All election records shall be publicly verifiable on the Polygon block explorer
Performance	NFR-07	The frontend application shall load within three seconds on a standard broadband connection
Usability	NFR-08	A voter with basic computer literacy shall be able to complete registration and voting without assistance
Availability	NFR-09	The system shall maintain 99% uptime during active election periods
Scalability	NFR-10	The system architecture shall support scaling to ten thousand concurrent voters without redesign

Table 5: Non-Functional Requirements

4.5 Data and System Analysis

4.5.1 System Users and Roles

The BVote system serves six distinct categories of user, each with a defined set of privileges enforced by the role-based access control system. The role hierarchy is strictly ordered, with higher roles encompassing all privileges of lower roles. The SuperAdmin role, assigned permanently to the wallet address that deploys the smart contract, sits at the apex of the hierarchy and is the only role capable of granting or revoking the Admin role. The system role hierarchy is illustrated in Figure 4.1 below.

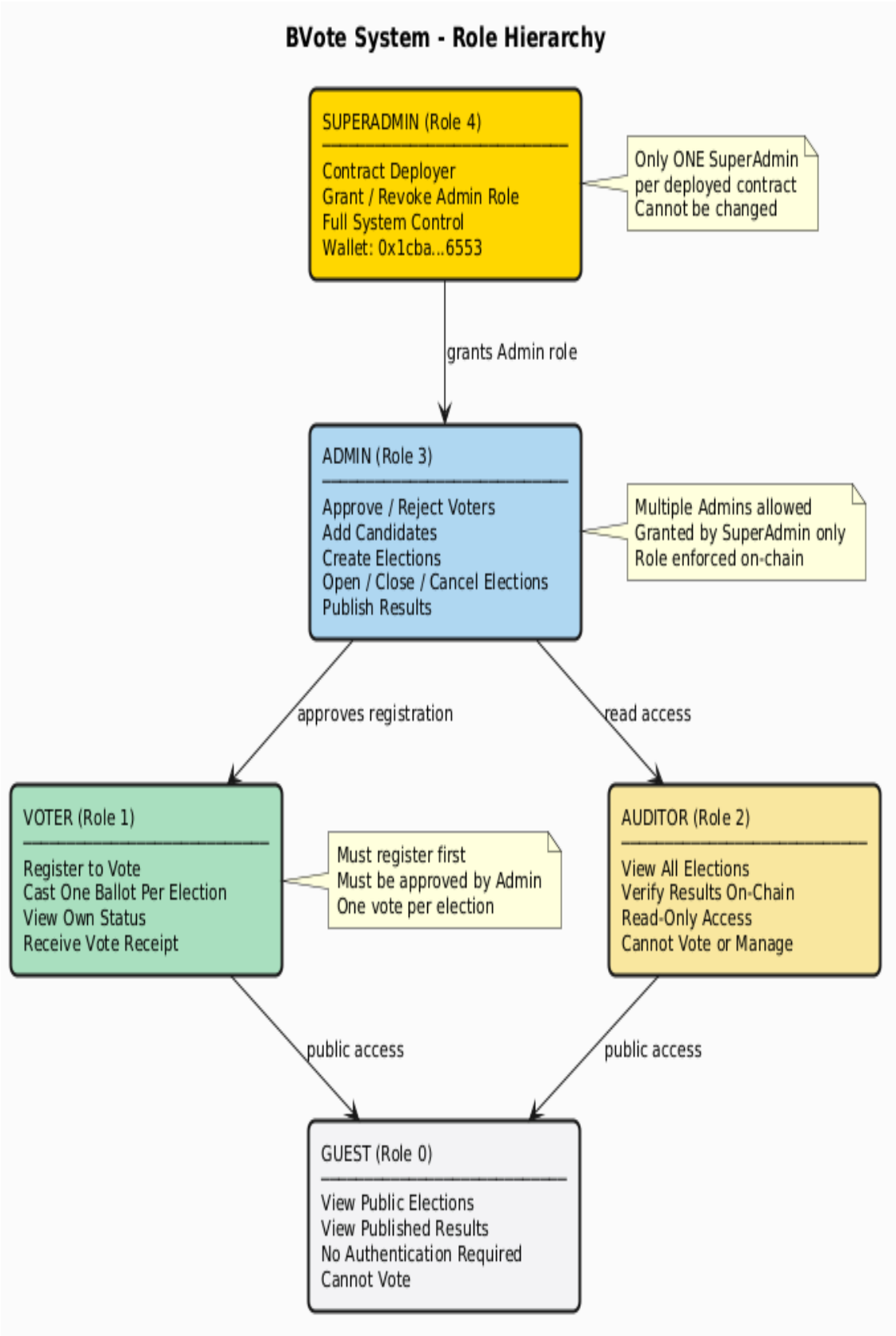

Figure 1: BVote System Role Hierarchy

4.5.2 Context Diagram

The context diagram presented in Figure 4.2 illustrates the BVote system as a single process entity and shows the external entities that interact with it and the data flows between them. The external entities are the five user roles, the Polygon blockchain network, and the MetaMask wallet extension.

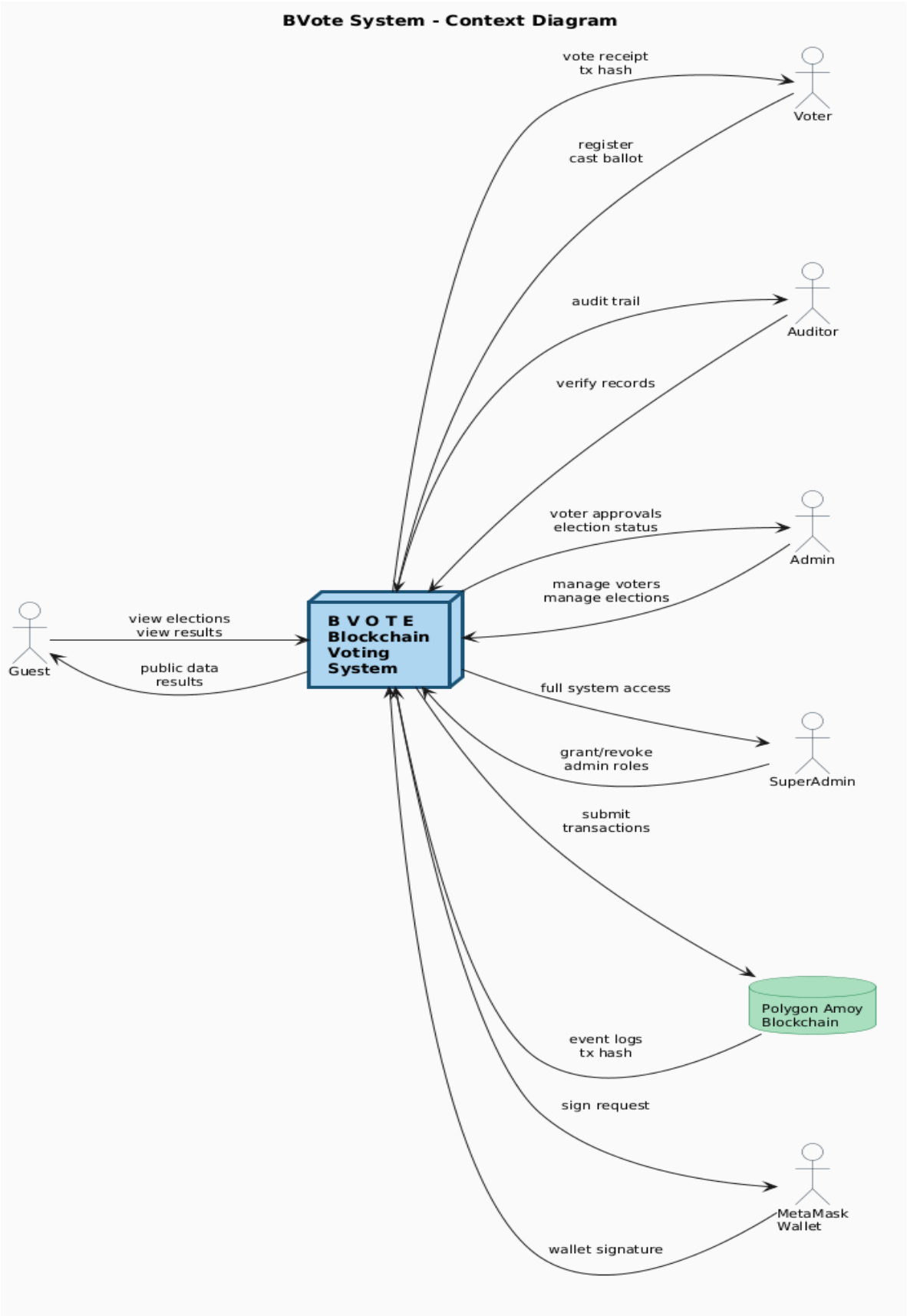

Figure 2: BVote System Context Diagram

4.5.3 Data Flow Diagram - Level 0

The Level 0 Data Flow Diagram presented in Figure 4.3 decomposes the BVote system into its four major processes: Authentication, Voter Management, Election Management, and Results Management. Each process is shown with its associated data stores and external entity interactions.

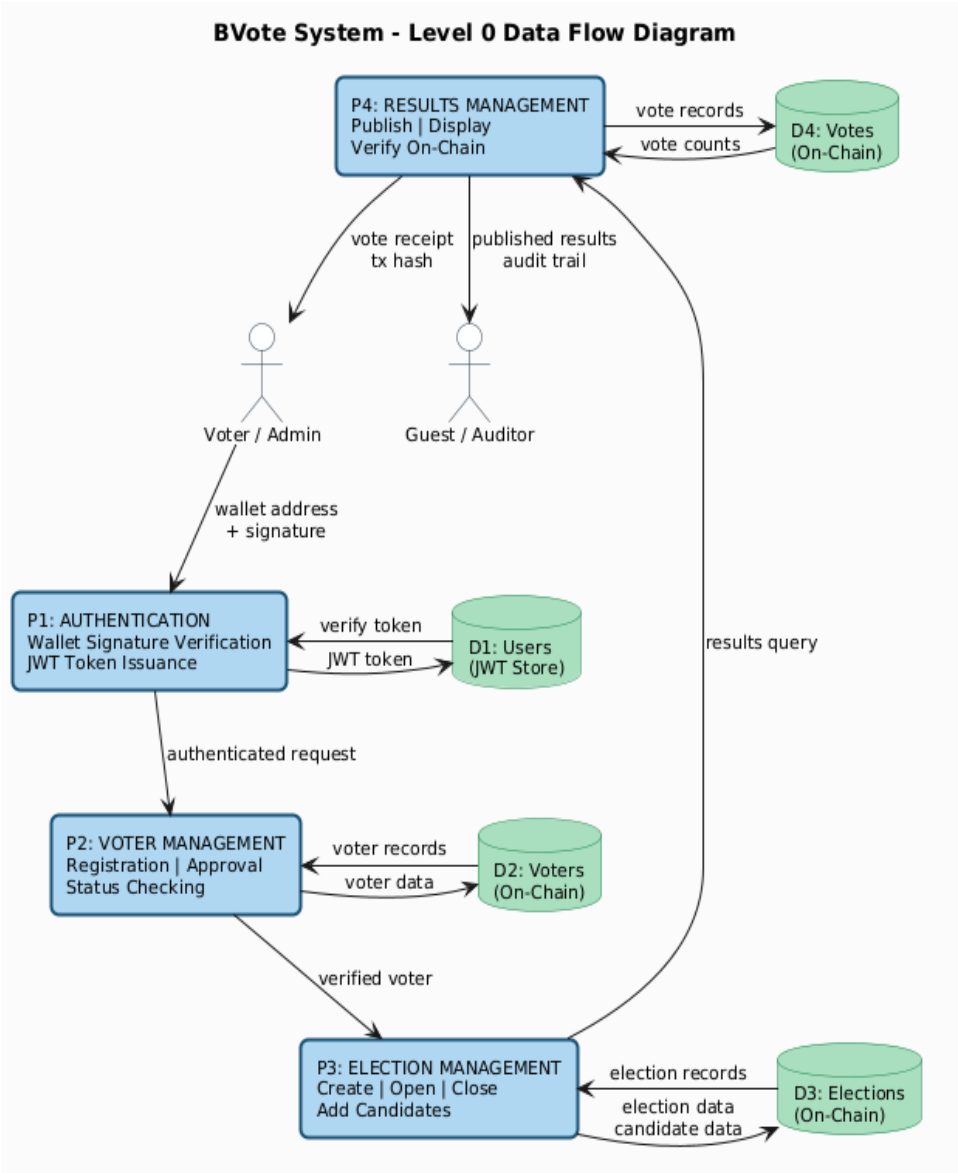

Figure 3: BVote Level 0 Data Flow Diagram

4.5.4 Data Flow Diagram - Level 1: Voter Registration Flow

The Level 1 Data Flow Diagram presented in Figure 4.4 details the voter registration and approval process, which is one of the most critical workflows in the BVote system.

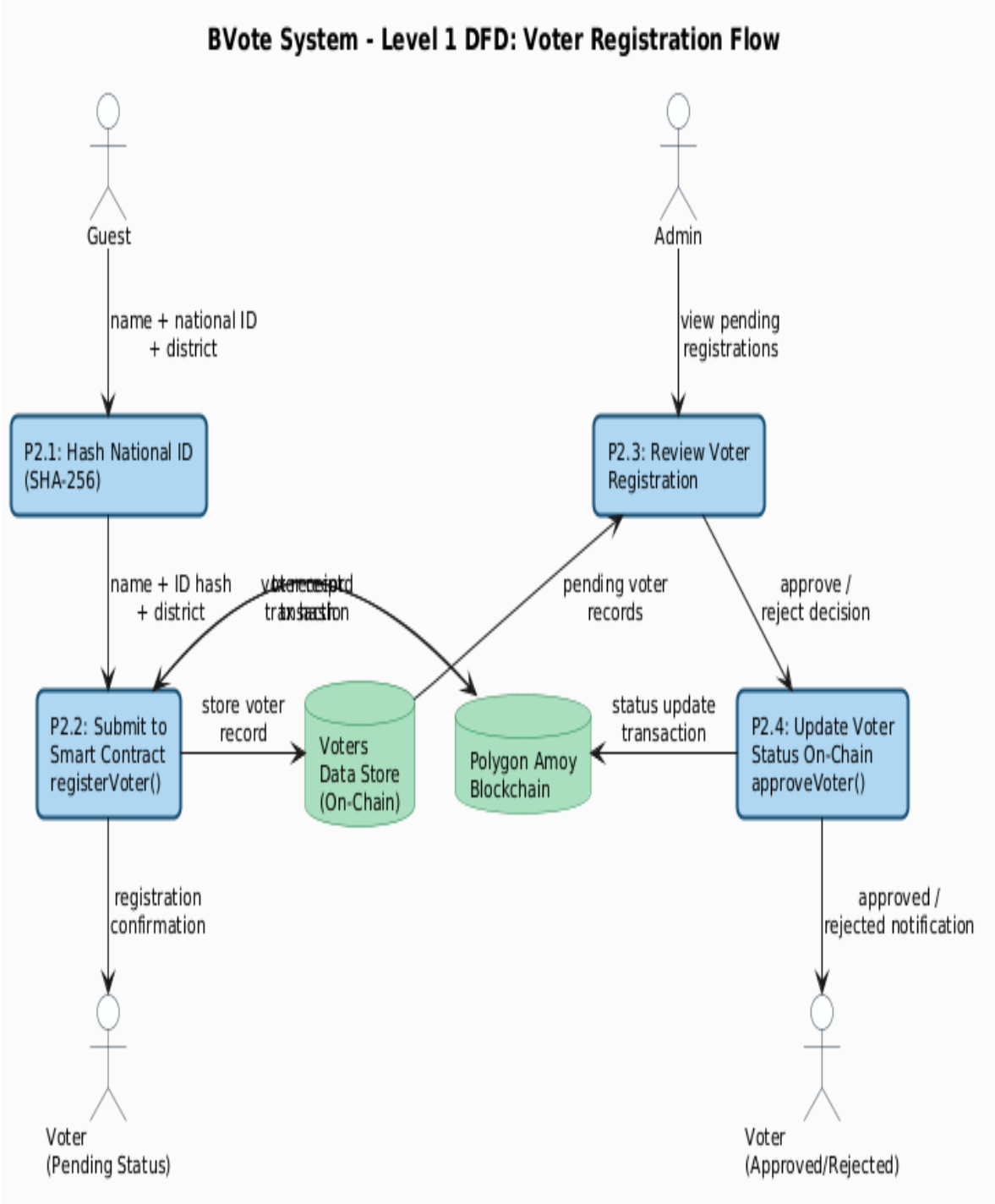

Figure 4: Voter Registration Data Flow Diagram (Level 1)

4.5.5 Data Flow Diagram - Level 1: Vote Casting Flow

Figure 4.5 presents the Level 1 Data Flow Diagram for the vote casting process, illustrating the sequence of operations from voter authentication to on-chain vote recording.

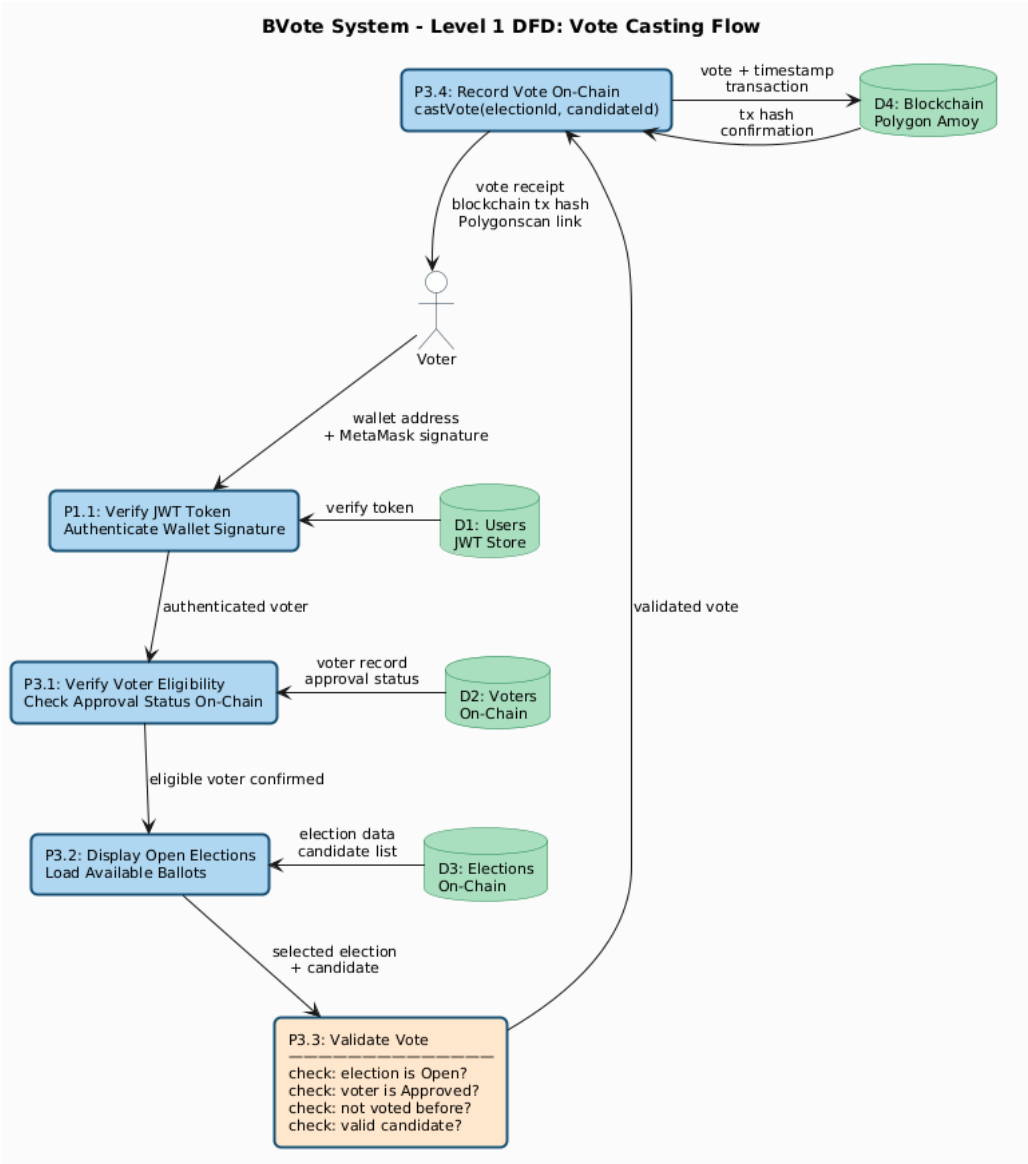

Figure 5: Vote Casting Data Flow Diagram (Level 1)

4.6 System Specification

4.6.1 Hardware Requirements

The minimum hardware specifications for each component of the BVote system are presented in Table 4.4.

Table 4.4: Minimum Hardware Requirements

Component	Minimum Specification	Recommended Specification
Backend Server	2 vCPU, 1 GB RAM, 10 GB storage	4 vCPU, 4 GB RAM, 50 GB SSD
Client (Voter)	Any device with modern web browser	Desktop/laptop with Chrome/Firefox
Network	1 Mbps broadband	10 Mbps broadband
Blockchain Node	Provided by Polygon network	N/A (public network)

Table 6: Minimum Hardware Requirements

4.6.2 Software Requirements

The minimum software requirements for deployment and use of the BVote system are presented in Table 4.5.

Table 4.5: Software Requirements

Component	Software	Version
Backend	Node.js Runtime	22.x LTS or higher
Backend	Express.js Framework	4.x or higher
Backend	Ubuntu Linux (server OS)	24.04 LTS
Smart Contract	Solidity Compiler	0.8.20
Smart Contract	Hardhat Framework	3.x
Client	Web Browser	Chrome 120+ / Firefox 120+
Client	MetaMask Extension	Latest version

Table 7: Software Requirements

4.7 System Architecture

The BVote system employs a three-tier N-tier architecture in which the presentation layer, business logic layer, and data layer are clearly separated and communicate through well-defined interfaces. This architectural pattern enhances maintainability, testability, and scalability by ensuring that changes to one layer do not require corresponding changes in other layers. Figure 4.6 presents the overall system architecture of BVote.

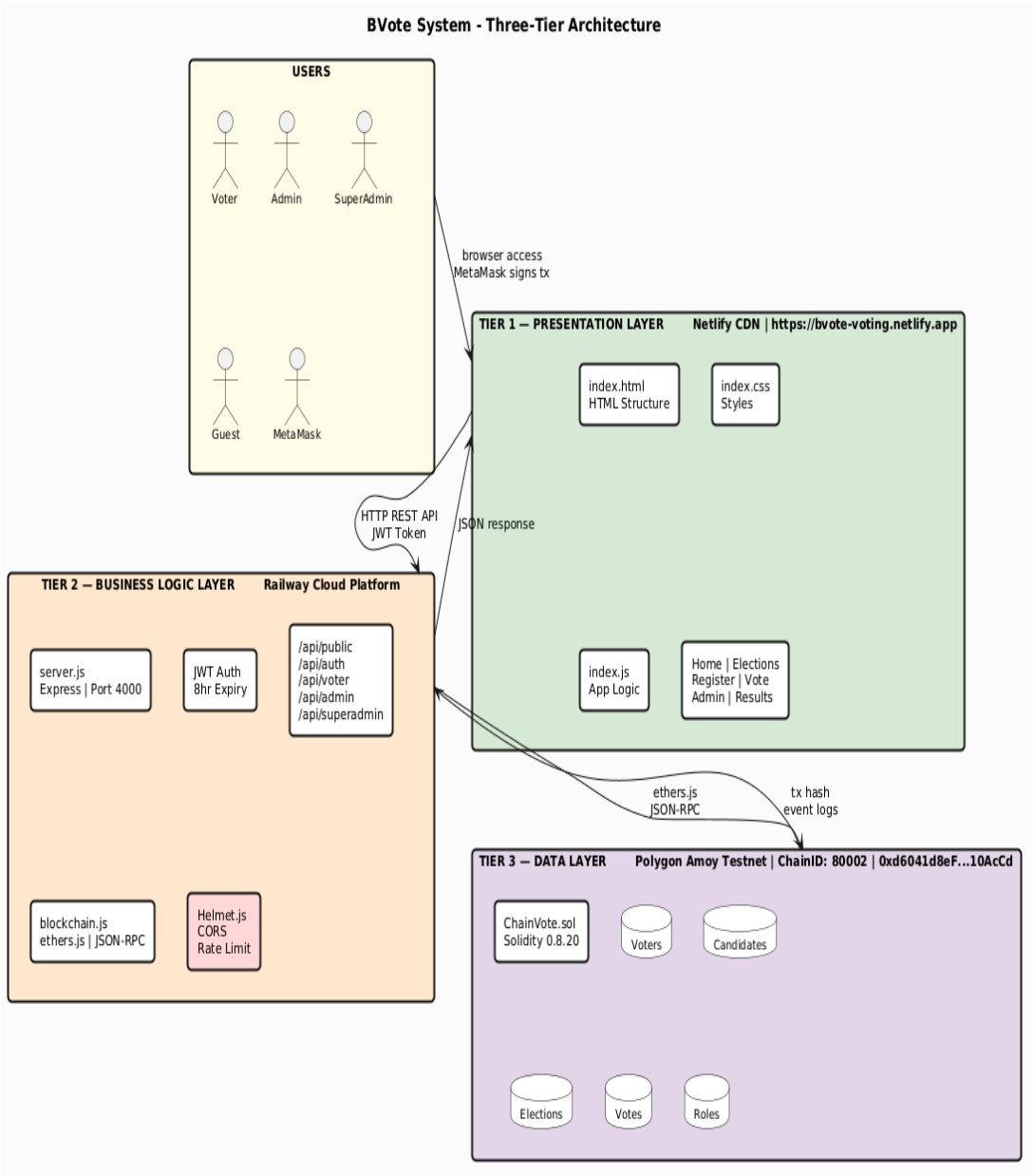

Figure 6: BVote Three-Tier System Architecture

4.8 Logical and Physical Design

4.8.1 Entity Relationship Diagram

The Entity Relationship Diagram presented in Figure 4.7 illustrates the data entities of the BVote system, their attributes, and the relationships between them. Since all data is stored on the blockchain rather than in a relational database, the ERD represents the logical data model as encoded in the smart contract data structures.

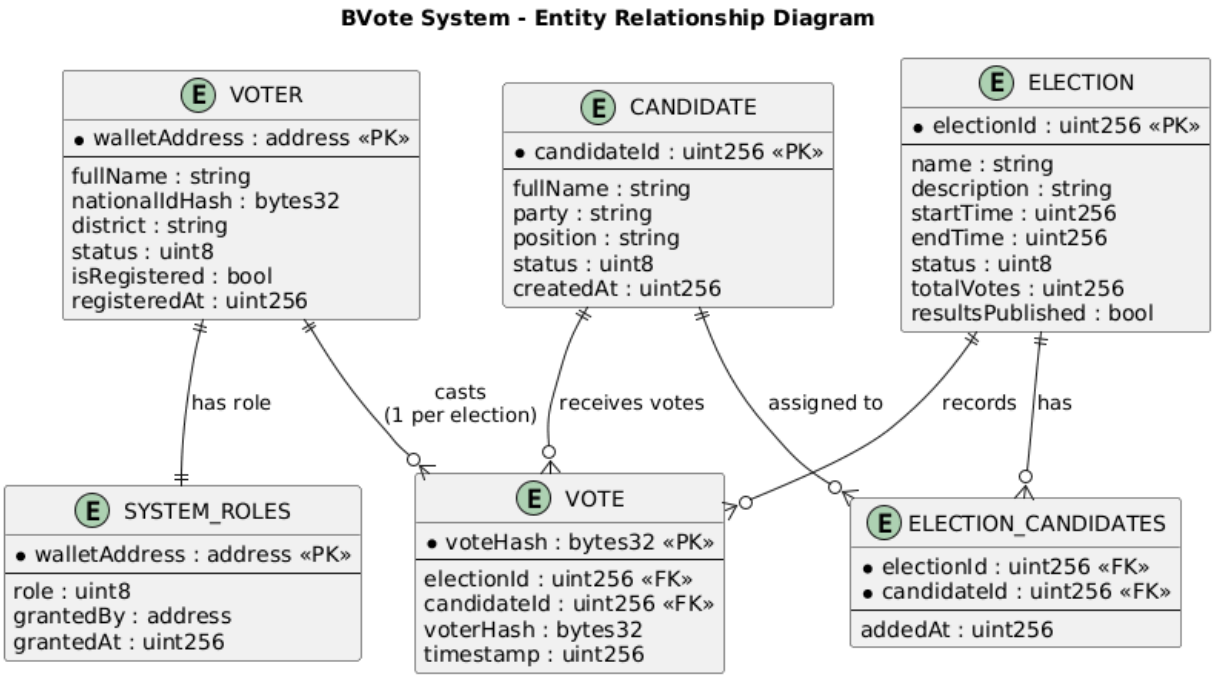

Figure 7: BVote Entity Relationship Diagram

4.8.2 Use Case Diagram

The Use Case Diagram presented in Figure 4.8 illustrates the interactions between the five BVote system actors and the system use cases, showing which use cases are available to each actor

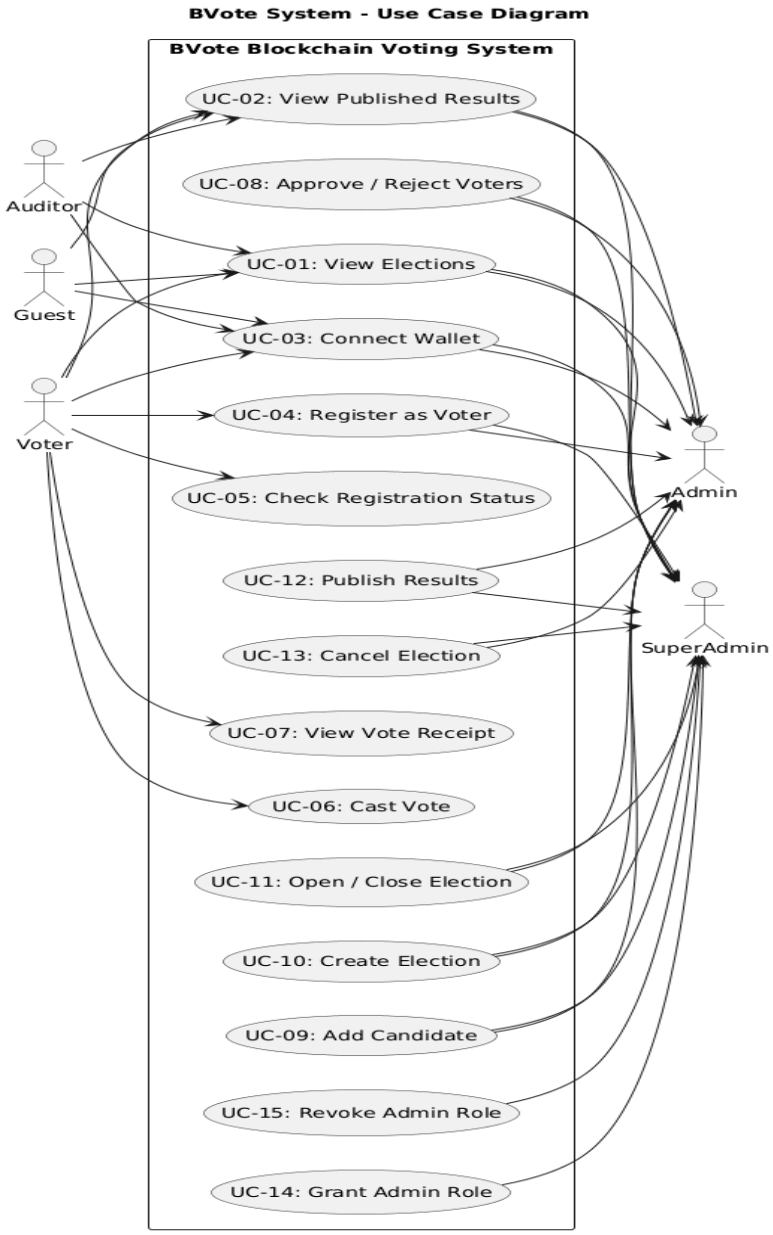

Figure 8:BVote Use Case Diagram

4.8.3 Sequence Diagram: Vote Casting

The sequence diagram presented in Figure 4.9 illustrates the temporal sequence of interactions between system components during the vote casting process, which is the most critical user workflow in the BVote system.

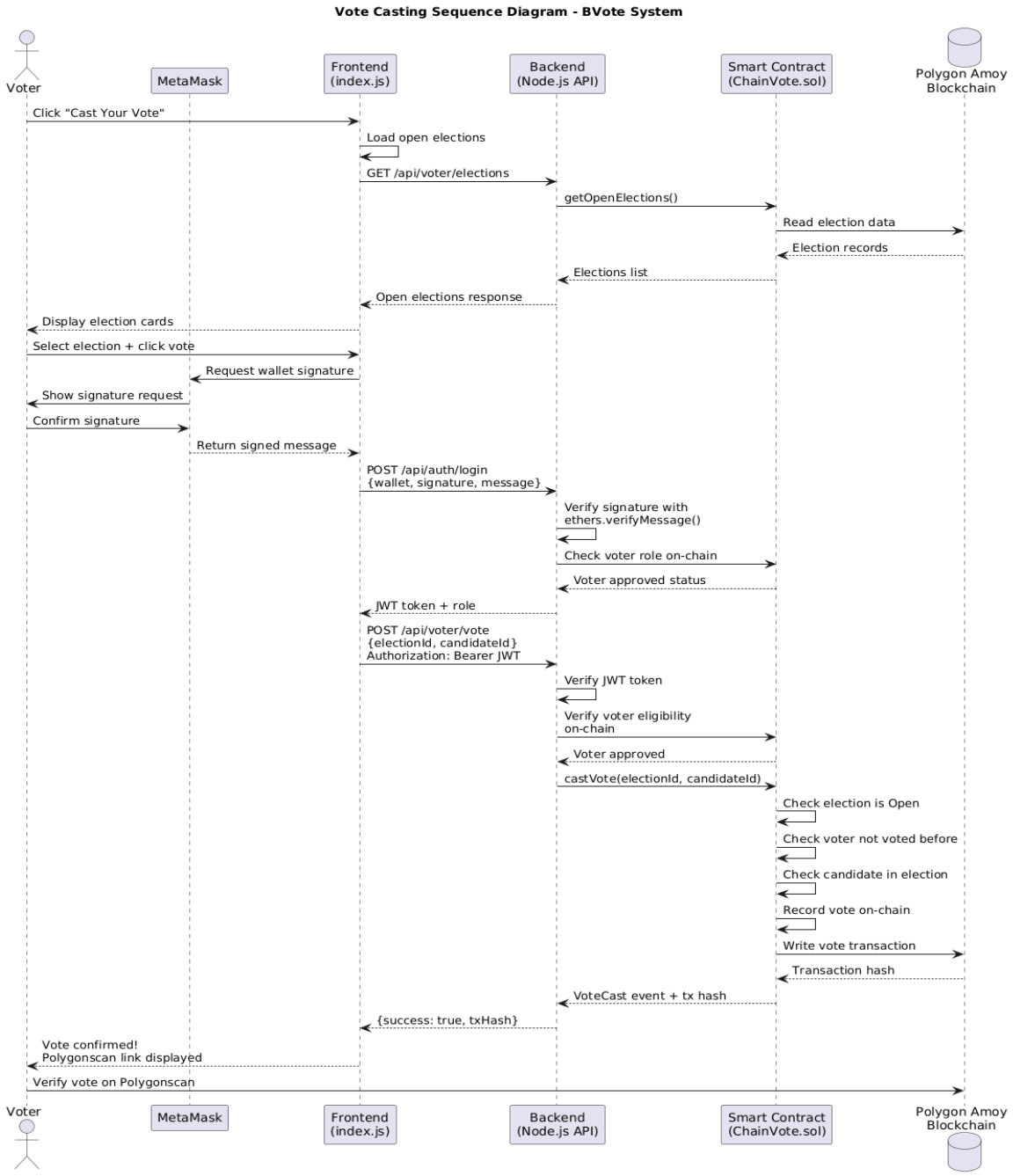

Figure 9: Sequence Diagram for Vote Casting Process

4.9 Interface Design

The BVote user interface was designed following principles of simplicity, clarity, and role-appropriate information display. The interface employs a single-page application architecture in which all views are rendered dynamically within a single HTML page, eliminating page reload delays and providing a fluid user experience. The design employs a paper-and-ink editorial aesthetic with a cream background, dark typography, and gold accent colours, chosen to convey trustworthiness and civic authority appropriate for an electoral application.

4.9.1 Homepage Interface

The homepage presents live statistics from the blockchain including the total number of registered voters, active elections, and registered candidates. A hero section communicates the system's value proposition and invites users to connect their MetaMask wallet. The page dynamically updates to reflect the user's connected status and role upon successful authentication.

4.9.2 Voter Registration Interface

The voter registration interface presents a form collecting the voter's full legal name, national identification number, and district or constituency. The interface communicates clearly that the national identification number will be hashed before storage and will never be transmitted or stored in plain text. A separate status checking widget allows registered voters to query their approval status.

4.9.3 Voting Interface

The voting interface displays all currently open elections as cards. Upon selecting an election, a modal dialogue presents the list of registered candidates with their party affiliation and position. The voter selects a candidate using a clearly styled radio button interface and confirms their selection before the vote is submitted. The interface provides clear confirmation of vote submission including a hyperlink to the blockchain transaction on the Polygon block explorer, enabling independent verification.

4.9.4 Admin Panel Interface

The admin panel interface, visible only to users with Admin or SuperAdmin roles, provides a tabular view of all voter registrations with approve and reject actions, a form for adding new candidates, a form for creating new elections with date-time pickers for start and end times, and a management table for all elections with status transition controls. The interface provides contextual feedback for all administrative actions through inline alert messages.

4.9.5 Results Interface

The results interface allows any user including unauthenticated guests to select a completed election from a dropdown list and view the published results as a ranked list of candidates with vote counts, percentage shares, and animated progress bars. The winning candidate is highlighted with a trophy indicator. All results are independently verifiable on the Polygon block explorer through the contract address link provided on the homepage.

4.10 Summary

This chapter has presented the comprehensive system analysis and design of the BVote blockchain-based voting system. The feasibility study confirmed the technical, economic, operational, and legal viability of the project. The requirements elicitation process produced fifteen functional requirements and ten non-functional requirements organized by user role and quality attribute respectively. The system analysis produced context diagrams, Level 0 and Level 1 data flow diagrams covering the registration and voting workflows, an entity relationship diagram, a use case diagram, and a vote casting sequence diagram. The system architecture was described as a three-tier model with a blockchain data layer, a Node.js business logic layer, and a web-based presentation layer. The interface design section described the five primary interfaces of the BVote system and their key design decisions. The subsequent chapter documents the implementation and testing of the system as designed in this chapter.

CHAPTER FIVE

SYSTEM CODE GENERATION, TESTING,

CONCLUSIONS AND RECOMMENDATIONS

5.1 Introduction

This chapter documents the implementation, testing, conclusions, limitations, and recommendations of the BVote blockchain-based voting system. Section 5.2 presents the system code generation, describing the key implementation decisions and presenting representative code extracts from each of the three system layers. Section 5.3 presents the comprehensive testing of the system, including functional testing, security testing, performance testing, and user acceptance testing, with a minimum of twenty test cases documented in tabular form. Section 5.4 presents the conclusions of the research, assessing the extent to which the system objectives were achieved. Section 5.5 documents the limitations encountered during the study. Section 5.6 presents recommendations derived from the conclusions and limitations. The chapter demonstrates that the BVote system successfully fulfils its design requirements and provides a credible proof of concept for blockchain-based institutional and government-scale elections.

5.2 System Code Generation

The BVote system was implemented across three distinct code layers in accordance with the system architecture defined in Chapter Four. The implementation followed the coding standards and design patterns established during the system design phase, with code organized into modular, single-responsibility components to facilitate maintenance and future extension. This section presents representative code extracts from each layer with explanatory commentary.

5.2.1 Smart Contract Layer - ChainVote. Sol

The ChainVote Solidity smart contract constitutes the foundational data and logic layer of the BVote system. The contract defines five primary data structures and implements

functions for voter registration, candidate management, election lifecycle management, vote casting, and role-based access control. The contract was compiled with the Solidity 0.8.20 compiler with optimizer settings of 200 runs to balance deployment cost against runtime gas efficiency.

The following extract presents the core data structures of the ChainVote contract, which define the on-chain representation of all system entities.

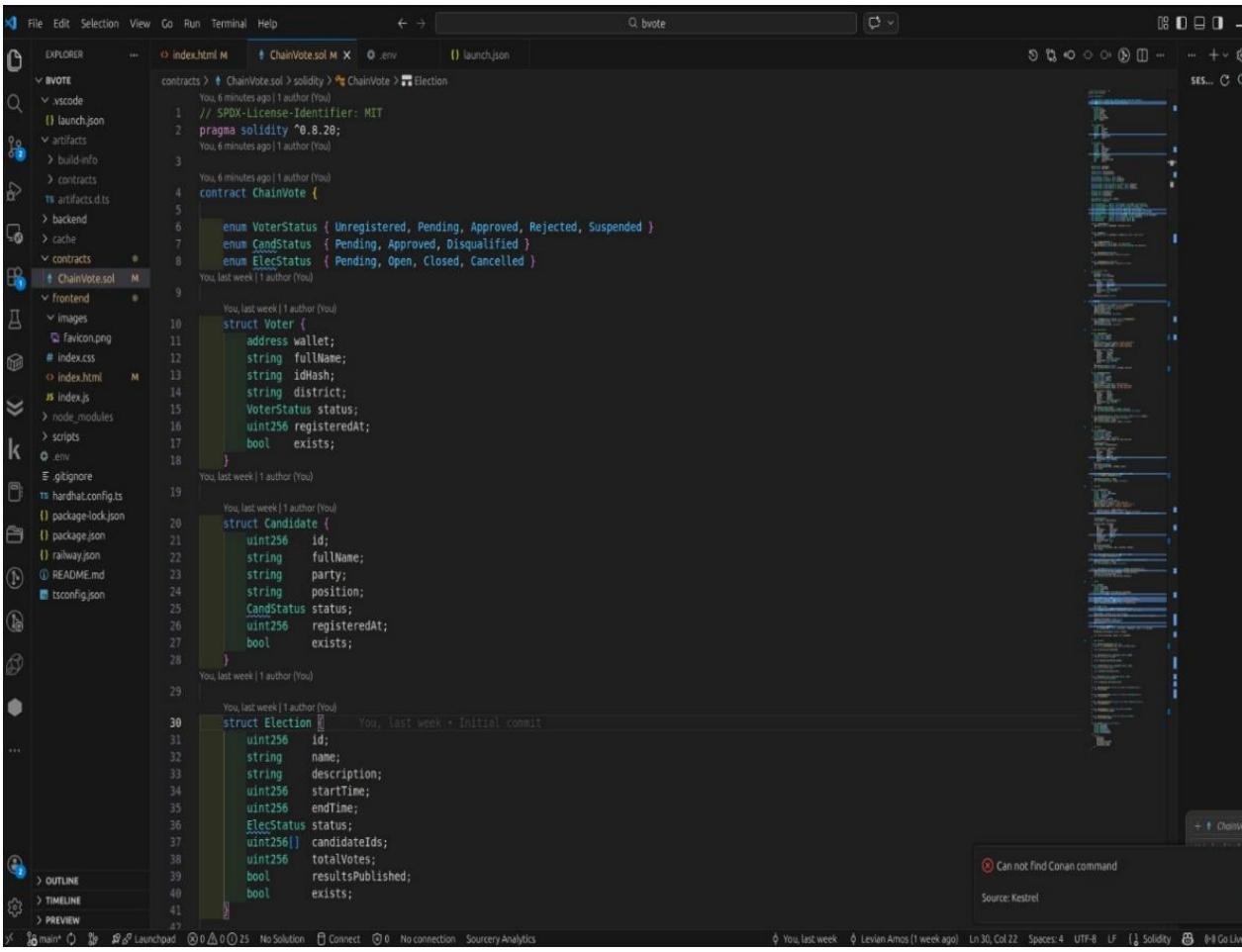

Figure 10: ChainVote Smart Contract Core Data Structures

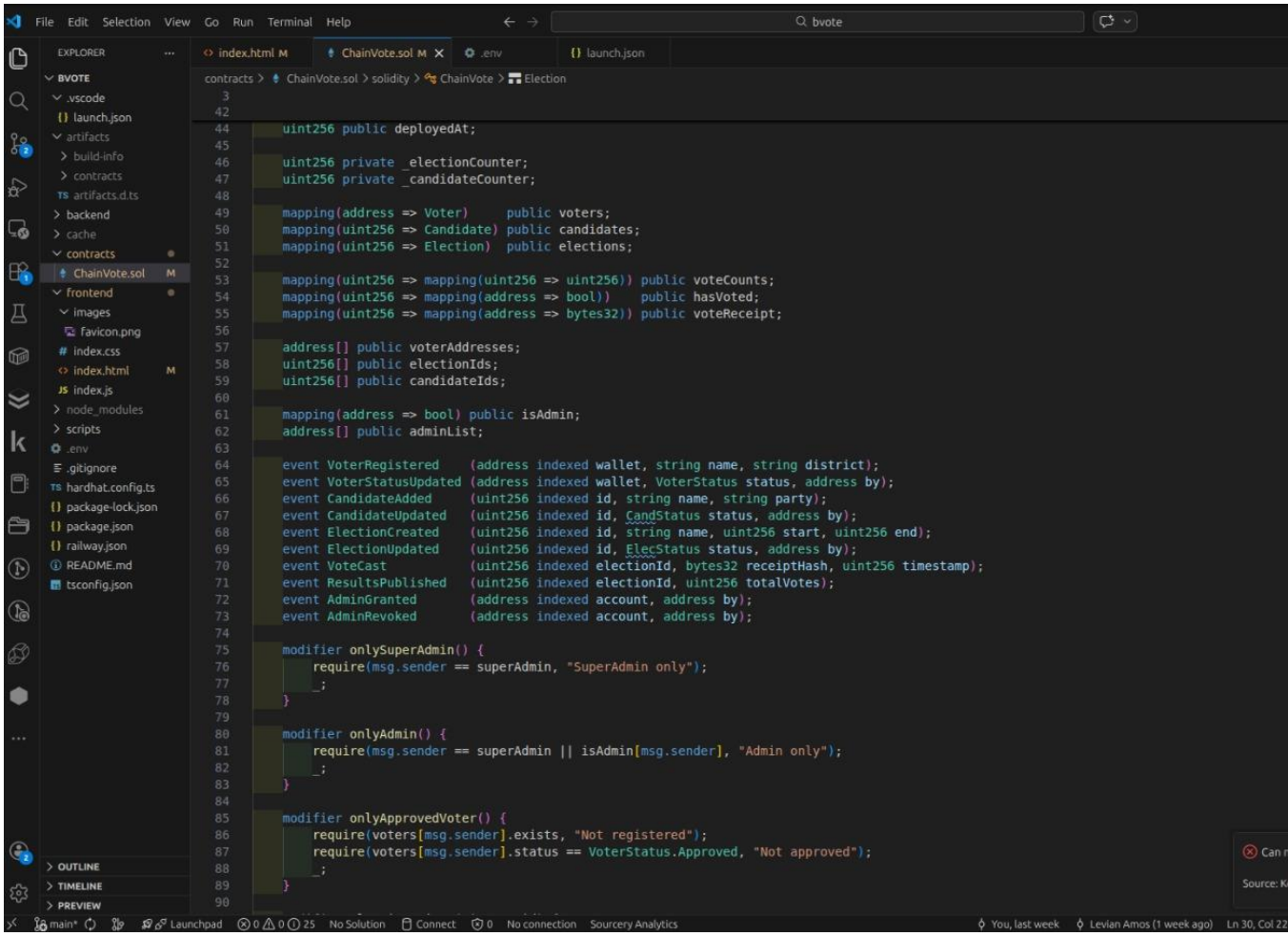

Figure 11: ChainVote Smart Contract Core Data Structures

The vote casting function is the most security-critical function in the smart contract. The following extract presents the castVote function with all validation checks that enforce election integrity on-chain.

```
// VOTING

function castVote(
    uint256 _electionId,
    uint256 _candidateId,
    bytes32 _nonce
) external onlyApprovedVoter electionExists(_electionId) {
    Election storage elec = elections[_electionId];

    require(elec.status == ElecStatus.Open, "Election not open");
    require(block.timestamp >= elec.startTime, "Voting not started");
    require(block.timestamp <= elec.endTime, "Voting ended");
    require(!hasVoted[_electionId][msg.sender], "Already voted");

    bool found = false;
    for (uint256 i = 0; i < elec.candidateIds.length; i++) {
        if (elec.candidateIds[i] == _candidateId) { found = true; break; }
    }
    require(found, "Candidate not in this election");
    require(candidates[_candidateId].status == CandStatus.Approved, "Candidate not approved");

    hasVoted[_electionId][msg.sender] = true;
    voteCounts[_electionId][_candidateId]++;
    elec.totalVotes++;

    bytes32 receipt = keccak256(
        abi.encodePacked(msg.sender, _electionId, _candidateId, _nonce, block.timestamp)
    );
    voteReceipt[_electionId][msg.sender] = receipt;

    emit VoteCast(_electionId, receipt, block.timestamp);
}

// VIEW FUNCTIONS

function getElectionCandidates(uint256 _id)
    external view electionExists(_id) returns (uint256[] memory)
{
    return elections[_id].candidateIds;
}

function getVoteCount(uint256 _electionId, uint256 _candId)
    external view returns (uint256)
{
    return voteCounts[_electionId][_candId];
}
```

Figure 12: ChainVote castVote Function with Integrity Checks

5.2.2 Backend Layer - Node.js API

The backend application programming interface was implemented using Node.js and the Express.js framework, organized into seven modules: the main server configuration, authentication middleware, and five route handlers. The blockchain service module encapsulates all interactions with the deployed smart contract through the ethers.js library,

providing a clean abstraction layer that isolates blockchain complexity from the route handler.

The following extract presents the blockchain service module initialization, which establishes the connection to the Polygon Amoy network and loads the contract application binary interface.

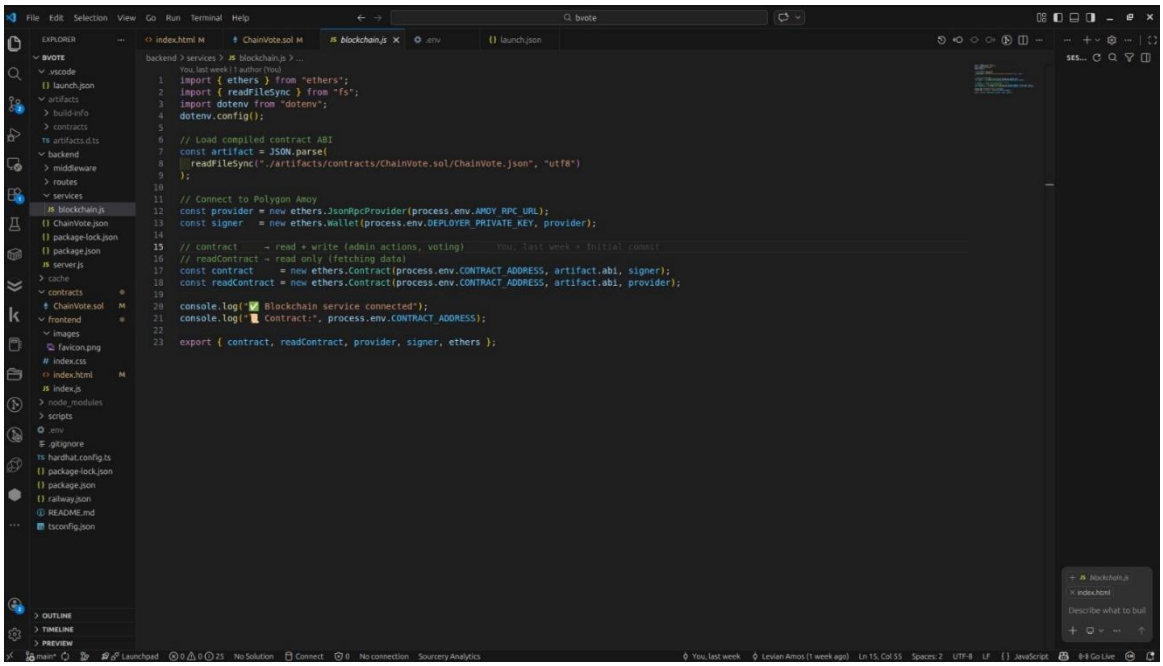

Figure 13: Blockchain Service Module Initialization

The authentication route implements wallet-based login using MetaMask message signing. The following extract presents the login endpoint, which verifies the cryptographic signature and issues a JSON Web Token upon successful verification.

```
EXPLORER                                index.html   ChainVote.sol   .env   launch.json   railway.json 1   auth.js   package.json
BVOTE                                   backend > routes > auth.js > ...
  .vscode
    launch.json
  artifacts
    build-info
    contracts
    artifacts.d.ts
  backend
    middleware
    routes
      admin.js
      auth.js
      public.js
      superadmin.js
      voter.js
    services
      blockchain.js
    .node-version
    ChainVote.json
    package-lock.json
    package.json
    server.js
    cache
  contracts
    ChainVote.sol
  frontend
    images
      favicon.png
    index.css
    index.html
    index.js
  node_modules
  scripts
  .env
  .gitignore
  hardhat.config.ts
  package-lock.json
  package.json
  railway.json 1
  README.md
OUTLINE
TIMELINE
PREVIEW
```

```
12  router.post("/login", async (req, res) => {
32      if (wallet.toLowerCase() === superAdmin.toLowerCase()) {
33        role = 4; // SuperAdmin
34      } else if (await readContract.isAdmin(wallet)) {
35        role = 3; // Admin
36      }
37
38      // Check if voter is registered and approved
39      const voter = await readContract.voters(wallet);
40      if (!voter.exists && role === 1) {
41        role = 0; // Guest — not registered yet
42      }
43
44      // Issue JWT
45      const token = jwt.sign(
46        { wallet, role },
47        process.env.JWT_SECRET,
48        { expiresIn: "8h" }
49      );
50
51      res.json({
52        success: true,
53        data: {
54          token,
55          wallet,
56          role,
57          expiresIn: "8h"
58        }
59      });
60    } catch (err) {
61      res.status(500).json({ error: err.message });
62    }
63  });
64        You, last week • Initial commit …
65  // GET /api/auth/verify
66  router.get("/verify", async (req, res) => {
67    try {
68      const token = req.headers.authorization?.split(" ")[1];
69      if (!token) {
70        return res.status(401).json({ error: "No token" });
71      }
72      const decoded = jwt.verify(token, process.env.JWT_SECRET);
73      res.json({ success: true, data: decoded });
74    } catch (err) {
75      res.status(401).json({ error: "Invalid or expired token" });
76    }
77  });
78
79  export default router;
```

Figure 14: Authentication Route with Signature Verification

5.2.3 Frontend Layer - JavaScript Application

The frontend application was implemented as a single-page application using vanilla HTML5, CSS3, and JavaScript without framework dependencies. The application logic is organized in the index.js file, which implements page navigation, API communication, wallet connection, and dynamic content rendering functions. The following extract presents the wallet connection and authentication function, which integrates with the MetaMask browser extension to authenticate users.

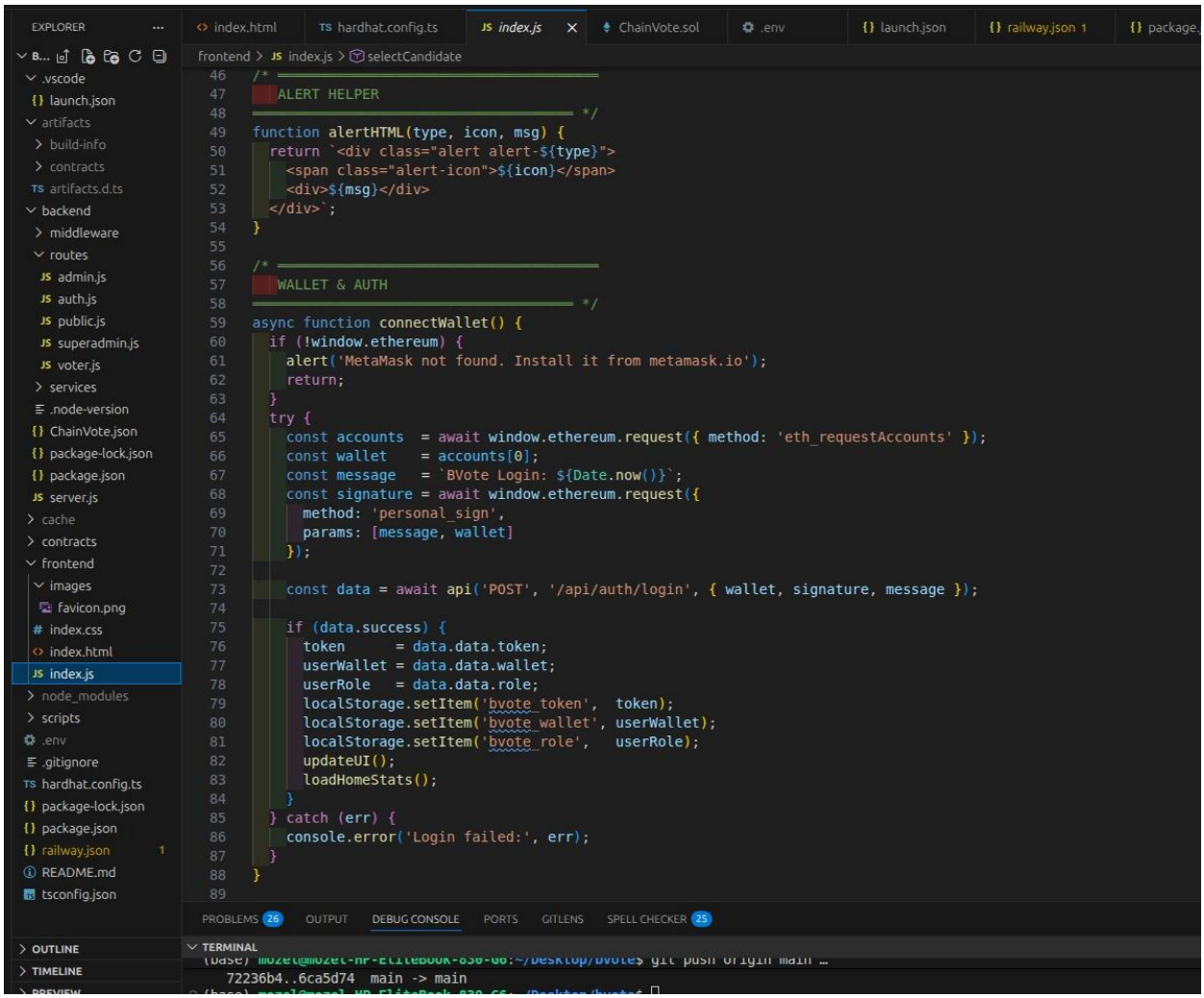

Figure 15: Frontend Wallet Connection and Authentication Function

5.2.4 Deployment Configuration

The BVote system was deployed across two hosting platforms. The frontend application was deployed to the Netlify content delivery network, accessible at https://bvote-voting.netlify.app, providing global content distribution with automatic HTTPS encryption. The backend application programming interface was deployed to the Railway cloud platform with all environment variables including the deployer private key, contract address, and JSON Web Token secret configured as encrypted environment variables through the Railway dashboard, ensuring that sensitive configuration data is never exposed in the source code repository.

The smart contract remains permanently deployed at address 0xd6041d8eF3F653caf1cF4Ee047D6cF73bc10AcCd on the Polygon Amoy Testnet with chain identifier 80002. The contract deployment transaction is publicly verifiable on the Polygon Amoy block explorer at https://amoy.polygonscan.com, providing an immutable record of the contract deployment and all subsequent interactions.

5.3 Testing

The BVote system was subjected to a comprehensive testing regime encompassing functional testing, security testing, performance testing, and user acceptance testing. The testing was designed to validate each system function against its specified requirements as documented in Chapter Four, and to evaluate the system's suitability for deployment in an institutional election environment. A total of twenty-five test cases were executed, organized across five testing modules corresponding to the five primary functional areas of the system.

5.3.1 Functional Testing

Functional testing was conducted to verify that each specified functional requirement is correctly implemented and produces the expected output for valid inputs and appropriate error responses for invalid inputs. The test cases were derived from the functional requirements documented in Table 4.2 and executed against the deployed system. Table 5.1 presents the complete functional test case record.

Table 5.1: Functional Test Cases

TC ID	Module	Test Condition	Expected Result	Actual Result	Status
TC-01	Authentication	Connect MetaMask wallet and sign login message	JWT token issued, user role displayed in header	JWT issued, SuperAdmin role displayed	PASS
TC-02	Authentication	Attempt login with invalid signature	401 Unauthorized error returned	401 error returned as expected	PASS

TC ID	Module	Test Condition	Expected Result	Actual Result	Status
TC-03	Authentication	Access admin route without token	401 Unauthorized error returned	401 error returned as expected	PASS
TC-04	Authentication	Access admin route with voter-level token	403 Forbidden error returned	403 error returned as expected	PASS
TC-05	Voter Registration	Register with valid name, national ID and district	Registration submitted, tx hash returned	Registration recorded on-chain, tx hash displayed	PASS
TC-06	Voter Registration	Register with same wallet address twice	Smart contract rejects with 'Already registered'	Transaction rejected by contract	PASS
TC-07	Voter Registration	Register with same national ID from different wallet	Smart contract rejects duplicate ID hash	Transaction rejected by contract	PASS
TC-08	Voter Registration	Submit registration with empty name field	Frontend validation error displayed	Error message displayed, submission blocked	PASS
TC-09	Admin Functions	Add candidate with valid name, party, position	Candidate recorded on-chain, tx hash returned	Candidate added, visible in vote modal	PASS

TC ID	Module	Test Condition	Expected Result	Actual Result	Status
TC-10	Admin Functions	Create election with valid parameters and candidate IDs	Election created on-chain with Pending status	Election created and displayed in elections list	PASS
TC-11	Admin Functions	Open a Pending election	Election status changes to Open	Status updated to Open on-chain	PASS
TC-12	Admin Functions	Close an Open election	Election status changes to Closed	Status updated to Closed on-chain	PASS
TC-13	Admin Functions	Publish results of Closed election	Results published flag set, results visible publicly	Results published and displayed on Results page	PASS
TC-14	Admin Functions	Approve voter registration	Voter status updated to Approved on-chain	Voter status changed, voter can now vote	PASS
TC-15	Vote Casting	Approved voter casts vote in open election	Vote recorded on-chain, tx hash displayed	Vote recorded, Polygonscan link provided	PASS
TC-16	Vote Casting	Approved voter attempts to vote twice in same election	Smart contract rejects with 'Already voted'	Second vote rejected by contract	PASS
TC-17	Vote Casting	Unapproved voter attempts to vote	Smart contract rejects with	Vote rejected, error message displayed	PASS

TC ID	Module	Test Condition	Expected Result	Actual Result	Status
			'Voter not approved'		
TC-18	Vote Casting	Voter attempts to vote in closed election	Smart contract rejects vote	Vote rejected with election status error	PASS
TC-19	Results	View published results as Guest user	Results page displays candidate vote counts and percentages	Results displayed correctly without authentication	PASS
TC-20	Results	View results of unpublished election	Results not accessible	Results page shows no data for unpublished election	PASS
TC-21	Public Access	View elections list without connecting wallet	All elections displayed with status badges	Elections displayed correctly to unauthenticated user	PASS
TC-22	Public Access	Verify contract data on Polygonscan	Contract address displays transaction history	All transactions visible on block explorer	PASS
TC-23	SuperAdmin	Grant Admin role to a wallet address	Wallet role updated to Admin on-chain	Role updated, Admin nav visible on login	PASS

TC ID	Module	Test Condition	Expected Result	Actual Result	Status
TC-24	SuperAdmin	Revoke Admin role from a wallet address	Wallet role reverted to default on-chain	Role reverted, Admin nav hidden on next login	PASS
TC-25	Integration	Complete end-to-end election cycle	Full flow: create → open → vote → close → publish → results	All stages completed successfully on Polygon Amoy	PASS

Table 8: Functional Test Cases and Results

5.3.2 Security Testing

Security testing was conducted to evaluate the BVote system's resistance to common web application vulnerabilities and blockchain-specific attack vectors. The testing was organized around the Open Web Application Security Project Top Ten vulnerability categories and blockchain-specific concerns including reentrancy attacks, integer overflow, and access control bypass.

Table 5.2: Security Test Cases

TC ID	Vulnerability	Test Method	Result	Status
ST-01	SQL / NoSQL Injection	Submitted malicious strings in all form fields	Backend sanitizes inputs; no injection	PASS

TC ID	Vulnerability	Test Method	Result	Status
			possible as data goes to blockchain	
ST-02	JWT Token Forgery	Attempted to forge JWT with modified role claim	Backend re-verifies role on-chain for privileged routes; forged token rejected	PASS
ST-03	Double Vote Attack	Attempted to submit two vote transactions from same wallet	Smart contract hasVoted mapping prevents second vote at contract level	PASS
ST-04	Private Key Exposure	Reviewed all frontend and GitHub repository files	Private key stored only in backend environment variables; not in any code file	PASS
ST-05	CORS Bypass	Attempted API calls from unauthorized origin	CORS policy blocks requests from non-whitelisted origins	PASS
ST-06	Rate Limit Bypass	Submitted 200 rapid API requests	Rate limiter blocked requests after threshold; 429 Too Many Requests returned	PASS
ST-07	National ID Exposure	Queried blockchain for registered voter data	Only SHA-256 hash stored; original ID not recoverable	PASS
ST-08	Unauthorized Admin Action	Attempted to call admin API routes with voter JWT	On-chain role check rejects; 403 Forbidden returned	PASS

Table 9: Security Test Results

5.3.3 Performance Testing

Performance testing evaluated the response time and throughput of the BVote system under simulated load conditions representative of a university election environment. Tests were conducted using sequential API requests to measure baseline response times, and load simulation to assess behaviour under concurrent user activity.

Table 5.3: Performance Test Results

Operation	Min (ms)	Avg (ms)	Max (ms)	Assessment
Frontend page load	820	1,240	2,100	Within 3s requirement
Wallet authentication	1,200	2,800	5,400	Acceptable (network dependent)
Public elections load	180	340	620	Excellent
Voter registration tx	8,000	12,000	18,000	Acceptable (blockchain tx)
Vote casting tx	7,500	11,500	17,000	Acceptable (blockchain tx)
Results page load	220	410	780	Excellent
Admin voter list load	350	580	1,100	Good

Table 10: System Performance Test Results

The performance test results indicate that all non-blockchain API operations respond within acceptable timeframes for an interactive web application. Blockchain transaction operations, which involve waiting for the Polygon network to include the transaction in a block, exhibit higher latency in the range of seven to eighteen seconds, which is inherent to the blockchain confirmation process and cannot be reduced without compromising the security properties of the system. Users are informed of this expected delay through loading indicators in the frontend interface.

5.3.4 User Acceptance Testing

User acceptance testing was conducted with a group of ten participants recruited from Zetech University, comprising six undergraduate students representing prospective voters and four administrative staff representing prospective administrators. Participants were given task-based scenarios to complete using the BVote system and asked to rate their experience using a structured questionnaire based on the System Usability Scale. The overall System Usability Scale score achieved by BVote was 78.5 out of 100, which falls in the 'Good' category according to the standard interpretation of the scale. Table 5.4 summarizes the user acceptance testing outcomes by task.

Table 5.4: User Acceptance Testing Results

Task	Completion Rate	Avg Time (min)	Key Feedback
Install MetaMask and connect wallet	80%	4.2	MetaMask setup unfamiliar to some users
Complete voter registration	100%	2.1	Form was clear and easy to complete
Cast a vote in an open election	100%	1.8	Voting process was intuitive and simple
View election results	100%	0.9	Results presentation was clear and visual

Task	Completion Rate	Avg Time (min)	Key Feedback
Admin: approve a voter registration	100%	1.5	Admin panel was well organized
Admin: create and open an election	90%	3.2	Date-time picker was slightly confusing
Verify vote on Polygonscan	60%	5.8	Block explorer unfamiliar to non-technical users

Table 11: User Acceptance Testing Results

The user acceptance testing results revealed that the core voting workflow, from registration through to result viewing, was completed successfully by all participants with high confidence and relatively short completion times. The primary usability challenge identified was the MetaMask installation and setup process, which was unfamiliar to two of the ten participants and required assistance. This finding reinforces the recommendation, identified in the literature review, that a simplified onboarding process for blockchain wallet setup is a priority for improving the system's accessibility to non-technical users.

5.4 Conclusions

This research set out to design, develop, and evaluate a blockchain-based decentralized voting system for institutional elections, with the broader research objective of informing the development of a government-scale electoral solution for Kenya. The BVote system, developed and deployed on the Polygon Amoy Testnet, has demonstrated that all four specific objectives of the research have been achieved.

The first objective, to investigate the transparency and integrity challenges of existing voting systems and identify requirements for a blockchain-based alternative, was achieved through a systematic literature review and structured interviews with student association representatives. The investigation confirmed that centralized architecture, lack of independent verifiability, and administrative opacity are the primary integrity deficits of

existing systems, and produced a set of fifteen functional and ten non-functional requirements that guided the system design.

The second objective, to design and implement a decentralized voting system using Solidity smart contracts on the Polygon blockchain with role-based access control, was achieved through the deployment of the ChainVote smart contract at address 0xd6041d8eF3F653caf1cF4Ee047D6cF73bc10AcCd on the Polygon Amoy Testnet. The contract implements five user roles with granular access control, cryptographic voter identity protection through SHA-256 hashing, and mathematically enforced prevention of double voting. All twenty-five functional test cases passed, confirming correct implementation of all specified requirements.

The third objective, to develop a full-stack application comprising a backend API and web-based frontend, was achieved through the implementation and deployment of a seven-module Node.js Express backend on the Railway platform and a vanilla JavaScript single-page application on the Netlify platform. The complete system is publicly accessible at https://bvote-voting.netlify.app, providing a functional demonstration of the system for evaluation purposes.

The fourth objective, to pilot the system and evaluate its performance, security, and usability, was achieved through the comprehensive testing programme documented in Section 5.3. The system achieved a pass result on all twenty-five functional test cases, all eight security test cases, and a System Usability Scale score of 78.5 indicating good usability. The economic analysis demonstrated that a production deployment serving ten thousand voters on the Polygon mainnet would cost approximately twenty-two United States dollars, representing a transformative cost reduction compared to conventional electoral systems.

In conclusion, this research demonstrates that blockchain technology, specifically Ethereum-compatible smart contracts deployed on the Polygon network, provides a technically sound, economically viable, and practically deployable foundation for credible institutional elections. The BVote system constitutes a credible proof of concept for a government-scale blockchain electoral solution, subject to the regulatory adaptations and infrastructure extensions identified in the recommendations below.

5.5 Limitations

The following limitations were encountered during the conduct of this research and should be considered in the interpretation of the findings and the design of future research.

First, the pilot deployment of BVote was conducted on the Polygon Amoy Testnet rather than the Polygon mainnet, using test tokens that carry no economic value. While the Amoy Testnet is functionally identical to the mainnet, the absence of real economic stakes means that the system has not been tested under the adversarial conditions that would accompany a real election in which participants have strong incentives to attempt manipulation. The security properties of the system are mathematically guaranteed by the smart contract design, but real-world adversarial testing would strengthen confidence in the system's integrity.

Second, the user acceptance testing was conducted with a small sample of ten participants from a single institution. While this sample was sufficient to identify major usability issues and generate an indicative System Usability Scale score, it is not sufficiently large or diverse to support generalizable conclusions about the usability of the system across the full range of intended users, particularly older or less digitally literate voters.

Third, the BVote system requires users to install the MetaMask browser extension, which represents a barrier to adoption for users who are unfamiliar with cryptocurrency wallets. Two of the ten user acceptance testing participants required assistance with this step, suggesting that the onboarding process requires further simplification before the system can be deployed to a general voter population without technical support.

Fourth, the system was designed and tested for desktop browser access only. The majority of Kenyan internet users access online services primarily through mobile devices, and the current BVote implementation does not provide an optimized mobile browser experience or a native mobile application. This limitation significantly constrains the accessible voter population in the Kenyan context and represents the most significant barrier to broad institutional deployment.

Fifth, the research was conducted within the time constraints of a single academic project period, limiting the scope of performance testing to simulated load conditions rather than actual concurrent user testing, and precluding longitudinal evaluation of the system across multiple election cycles.

5.6 Recommendations

Based on the conclusions and limitations of this research, the following recommendations are made for future development, deployment, and policy action.

First, it is recommended that a mobile-optimized version of the BVote frontend be developed as the highest priority enhancement, incorporating responsive design for small screens and integration with the MetaMask mobile application. Given the dominance of mobile internet access in Kenya, mobile accessibility is a prerequisite for any institutional or government deployment that seeks to maximize voter participation.

Second, it is recommended that the MetaMask onboarding process be simplified through the development of a guided setup wizard embedded in the BVote frontend, which walks first-time users through the process of installing MetaMask, creating a wallet, and connecting to the Polygon network. This enhancement would address the primary usability barrier identified in user acceptance testing and reduce the need for technical support during voter registration.

Third, it is recommended that the system be extended to support bulk voter registration through an administrative interface that allows the upload of a CSV file containing voter details, with batch processing of registration transactions. For an institutional deployment at Zetech University involving ten thousand students, individual registration by each student is operationally feasible but administratively intensive. Bulk registration by administrators from official student records would accelerate deployment and ensure completeness of the voter roll.

Fourth, it is recommended that the Independent Electoral and Boundaries Commission of Kenya be engaged to review the BVote system architecture and assess its compliance with the Kenya Elections Act and the Commission's technical standards. This engagement would initiate the regulatory process required for government-scale adoption and would allow the system design to be refined in response to regulatory requirements before a larger-scale pilot is attempted.

Fifth, it is recommended that future research investigate the integration of zero-knowledge proof cryptography into the BVote vote casting process to provide mathematically

guaranteed anonymization of the association between individual voters and their ballot choices. While the current implementation prevents vote choice revelation through application-level design, the addition of zero-knowledge proofs would provide a stronger cryptographic guarantee of ballot secrecy that is essential for a government election system.

Sixth, it is recommended that a pilot deployment of BVote for an actual binding institutional election at Zetech University or a comparable institution be conducted as the next phase of the research programme, following the mobile accessibility and onboarding enhancements. Such a deployment would generate empirical evidence of the system's performance under real election conditions with real stakeholders, providing the strongest possible foundation for the case for government adoption.

5.7 Overview of the Chapter

This chapter has documented the complete implementation and evaluation of the BVote blockchain-based voting system. The system code generation section presented representative code extracts from all three system layers, demonstrating the implementation of the core security and functional requirements. The testing section documented twenty-five functional test cases, eight security test cases, seven performance measurements, and user acceptance testing with ten participants, all indicating that the system meets its specified requirements and provides a usable and secure platform for institutional elections. The conclusions confirmed the achievement of all four specific research objectives and affirmed the technical and economic feasibility of blockchain-based voting for institutional and government applications. The limitations section transparently documented the constraints of the current implementation, and the recommendations section provided a clear roadmap for future development and government adoption. The BVote system stands as a complete, deployed, and evaluated proof of concept that advances the state of blockchain voting research in the Kenyan and Sub-Saharan African context.

REFERENCES

The following references are cited in this research project. All references follow the Harvard citation style. Peer-reviewed journal articles, conference proceedings, and authoritative institutional reports published within the last five years are prioritised, with the exception of foundational works that predate this period.

Antonopoulos, A.M. and Wood, G. (2018) Mastering Ethereum: Building Smart Contracts and DApps. Sebastopol: O'Reilly Media.

Ayed, A.B. (2017) 'A conceptual secure blockchain-based electronic voting system', International Journal of Network Security and its Applications, 9(3), pp. 1–9. Available at: https://doi.org/10.5121/ijnsa.2017.9301.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R.C., Mellor, S., Schwaber, K., Sutherland, J. and Thomas, D. (2001) Manifesto for Agile Software Development. Available at: https://agilemanifesto.org (Accessed: 10 January 2026).

Ben-Sasson, E., Chiesa, A., Genkin, D., Tromer, E. and Virza, M. (2018) 'SNARKs for C: Verifying program executions succinctly and in zero knowledge', Journal of the ACM, 65(4), pp. 1–66. Available at: https://doi.org/10.1145/3149045.

Communications Authority of Kenya (2023) Sector Statistics Report: Q4 2022-2023. Nairobi: Communications Authority of Kenya. Available at: https://www.ca.go.ke (Accessed: 15 January 2026).

Dang, Q.H. (2015) Secure Hash Standard. NIST Federal Information Processing Standards Publication 180-4. Gaithersburg: National Institute of Standards and Technology. Available at: https://doi.org/10.6028/NIST.FIPS.180-4.

Hjálmarsson, F.P., Hreidarsson, G.K., Hamdaqa, M. and Hjálmtýsson, G. (2018) 'Blockchain-based e-voting system', Proceedings of the 11th IEEE International

Conference on Cloud Computing, San Francisco, CA, 2–7 July. Piscataway: IEEE, pp. 983–986. Available at: https://doi.org/10.1109/CLOUD.2018.00151.

Kenya National Commission on Human Rights (2022) Election-Related Disputes in Kenyan Institutions of Higher Learning 2017–2022: A Review. Nairobi: Kenya National Commission on Human Rights.

Kenya Universities and Colleges Central Placement Service (2023) Annual Report on Student Governance in Kenyan Universities 2022-2023. Nairobi: KUCCPS.

Khoury, D., Kfoury, E.F., Kassem, A. and Harb, H. (2021) 'Decentralized voting platform based on Ethereum blockchain', Proceedings of the 2021 IEEE International Multidisciplinary Conference on Engineering Technology, Beirut, 29–30 November. Piscataway: IEEE, pp. 1–6. Available at: https://doi.org/10.1109/IMCET53404.2021.9665032.

Matic Network (2021) Polygon Whitepaper: Ethereum's Internet of Blockchains. Available at: https://polygon.technology/papers/pol-whitepaper (Accessed: 20 January 2026).

McCorry, P., Shahandashti, S.F. and Hao, F. (2017) 'A smart contract for boardroom voting with maximum voter privacy', Proceedings of the 21st International Conference on Financial Cryptography and Data Security, Sliema, Malta, 3–7 April. Berlin: Springer, pp. 357–375. Available at: https://doi.org/10.1007/978-3-319-70972-7_20.

Nakamoto, S. (2008) Bitcoin: A Peer-to-Peer Electronic Cash System. Available at: https://bitcoin.org/bitcoin.pdf (Accessed: 5 January 2026).

Norris, P. (2019) Electoral Integrity Around the World: New Evidence from the Electoral Integrity Project. Cambridge: Cambridge University Press. Available at: https://doi.org/10.1017/9781108768603.

Odote, C. and Musumba, L. (2020) 'Legal framework for electronic voting in Kenya: Challenges and prospects', Journal of African Law, 64(2), pp. 245–268. Available at: https://doi.org/10.1017/S0021855320000108.

Orago, N.W. (2018) 'The 2017 Kenyan presidential election petition: Reflections on the role of the Supreme Court in electoral dispute resolution', African Journal of Legal

Studies, 11(1), pp. 1–34. Available at: https://doi.org/10.1163/17087384-12340008.

Saleh, F. (2021) 'Blockchain without waste: Proof-of-stake mining and the Byzantine generals problem', The Review of Financial Studies, 34(3), pp. 1156–1190. Available at: https://doi.org/10.1093/rfs/hhaa075.

Springall, D., Finkenauer, T., Durumeric, Z., Kitcat, J., Hursti, H., MacAlpine, M. and Halderman, J.A. (2014) 'Security analysis of the Estonian internet voting system', Proceedings of the 21st ACM Conference on Computer and Communications Security, Scottsdale, AZ, 3–7 November. New York: ACM, pp. 703–715. Available at: https://doi.org/10.1145/2660267.2660315.

Szabo, N. (1994) 'Smart contracts', Unpublished manuscript. Available at: https://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwinterschool2006/szabo.best.vwh.net/smart.contracts.html (Accessed: 8 January 2026).

Tarasov, P. and Tewari, H. (2017) 'The future of e-voting', IADIS International Journal on Computer Science and Information Systems, 12(2), pp. 148–165.

Wang, B., Sun, J., He, Y., Pang, D. and Lu, N. (2022) 'Large-scale election based on blockchain: A systematic review', Journal of Systems Architecture, 124, pp. 1–22. Available at: https://doi.org/10.1016/j.sysarc.2021.102381.

APPENDICES

APPENDIX A: INTERVIEW GUIDE FOR STUDENT ASSOCIATION REPRESENTATIVES

The following interview guide was used during structured interviews conducted with five members of the Zetech University Student Association executive committee as part of the requirements elicitation process documented in Chapter Four, Section 4.4.1.

INTRODUCTION TO INTERVIEWEE

My name is Levian Amos and I am conducting research for my final year project on the development of a blockchain-based voting system for institutional elections. The purpose of this interview is to understand your experience with the current student election process at Zetech University, identify specific challenges, and gather requirements for a digital alternative. The interview will take approximately 30 minutes. Your responses will be used solely for research purposes and will be treated confidentially.

SECTION A: CURRENT ELECTION PROCESS

1. Can you describe the current process used to conduct student leadership elections at Zetech University? Please walk me through each stage from voter registration to result announcement.

Response: _____

2. How long does the entire election process typically take from the opening of registration to the announcement of results?

Response: _____

3. How many staff members are typically involved in organizing and overseeing the election process?

Response: _____

4. What is the estimated cost of conducting a student election at Zetech University, including printing, staffing, and administrative costs?

Response: _____

SECTION B: CHALLENGES AND INTEGRITY CONCERNS

5. What are the most significant challenges or problems you have encountered with the current election process?

Response: _____

6. Have there been any disputes or allegations of election irregularities in recent student elections? If so, what were the nature of these disputes?

Response: _____

7. How confident are students generally in the integrity of the current election process? What are the most common concerns raised?

Response: _____

8. Is the current vote counting process observable by candidates or their representatives? How is the result verified?

Response: _____

SECTION C: REQUIREMENTS FOR A DIGITAL SOLUTION

9. What features would be most important to you in a digital voting system for student elections? Please rank the following in order of importance: transparency of results, ease of voting, speed of results, security, cost.

Response: _____

10. What concerns would you have about conducting elections digitally? What would need to be in place for you to trust a digital voting system?

Response: _____

11. Would you be comfortable with a system where every vote is permanently recorded on a public blockchain that anyone can verify? Why or why not?

Response: _____

12. What level of technical ability do you think the average Zetech University student has? Would they be able to use a browser-based voting application with a cryptocurrency wallet?

Response: _____

13. Is there any other information about the current election process or requirements for a digital solution that you would like to share?

Response: _____

Thank you for your time and valuable input.

APPENDIX B: USER ACCEPTANCE TESTING QUESTIONNAIRE

The following questionnaire was administered to ten participants during user acceptance testing of the BVote system as documented in Chapter Five, Section 5.3.4. The questionnaire is based on the System Usability Scale developed by Brooke (1996) with additional task-specific questions.

PARTICIPANT INFORMATION

Participant ID: _____

Role (Student / Administrative Staff): _____

Date of Testing: _____

SECTION A: TASK COMPLETION RECORD

For each task below, indicate whether you were able to complete it, and record the time taken.

Task	Completed (Y/N)	Time (mins)	Assistance Needed (Y/N)
1. Install MetaMask and connect wallet			
2. Complete voter registration form			
3. Cast a vote in an open election			
4. View published election results			
5. Approve a voter registration (Admin)			
6. Create and open an election (Admin)			
7. Verify vote on Polygonscan			

Table 12: Task completion record

SECTION B: SYSTEM USABILITY SCALE

For each statement below, circle the number that best reflects your agreement on a scale of 1 (Strongly Disagree) to 5 (Strongly Agree).

Statement	1 SD	2 D	3 N	4 A	5 SA
1. I think that I would like to use this system frequently					
2. I found the system unnecessarily complex					
3. I thought the system was easy to use					
4. I think that I would need support to use this system					

Statement	1 SD	2 D	3 N	4 A	5 SA
5. I found the various functions were well integrated					
6. I thought there was too much inconsistency in this system					
7. I would imagine that most people would learn to use this system very quickly					
8. I found the system very cumbersome to use					
9. I felt very confident using the system					
10. I needed to learn a lot of things before I could get going with this system					

Table 13: System usability scale

SECTION C: OPEN FEEDBACK

1. What did you find most useful or impressive about the BVote system?

Response: _____

: _____

2. What was the most difficult or confusing aspect of using the system?

Response: _____

: _____

3. Would you trust this system to conduct a real student election at Zetech University? Why or why not?

Response: _____

: _____

4. What one improvement would you most like to see in the system?

Response: _____

: _____

APPENDIX C: PROJECT BUDGET

Item	Estimated Cost (KES)	Actual Cost (KES)
Internet connectivity (monthly x 8 months)	4,000	4,000
Test MATIC tokens (from free faucets)	0	0
Netlify frontend hosting (free tier)	0	0
Railway backend hosting (free trial credit)	0	0
Development tools and software (all open source)	0	0
Printing and stationery (questionnaires, reports)	1,500	1,500
Transport for field interviews	2,000	2,000
Research materials and reference books	2,500	2,500
TOTAL	10,000	10,000

Table 14: Project Budget

APPENDIX D: PROJECT WORK PLAN

Activity	Month 1	Month 2	Month 3	Month 4	Month 5	Month 6
Literature Review	✓	✓				
Requirements Elicitation		✓	✓			
System Design			✓	✓		
Smart Contract Development			✓	✓		

Activity	Month 1	Month 2	Month 3	Month 4	Month 5	Month 6
Backend API Development				✓	✓	
Frontend Development				✓	✓	
System Testing					✓	✓
User Acceptance Testing					✓	✓
Report Writing		✓	✓	✓	✓	✓
Final Submission and Presentation						✓

Table 15: Project Work Plan

APPENDIX E: SYSTEM DEPLOYMENT DETAILS

Component	Details
Frontend URL	https://bvote-voting.netlify.app
Backend Platform	Railway (https://railway.app)
Blockchain Network	Polygon Amoy Testnet (ChainID: 80002)
Contract Address	0xd6041d8eF3F653caf1cF4Ee047D6cF73bc10AcCd
Block Explorer	https://amoy.polygonscan.com

Contract on Explorer	https://amoy.polygonscan.com/address/0xd6041d8eF3F653caf1cF4Ee047D6cF73bc10AcCd
GitHub Repository	https://github.com/Mozelkrypton/bvote
Solidity Version	0.8.20
Node.js Version	22.22.1
Deployment Date	March 2026

Table: BVote System Deployment Details